HACKEN

Blockchain Protocol Security Analysis Report



Customer: Xinfin

Date: 13/10/2026

We express our gratitude to the Xinfin team for the collaborative engagement that enabled the execution of this Blockchain Protocol Security Assessment.

XDC Network is the execution client, a fork of go-ethereum v1.17.5 for XDC mainnet and the Apothem testnet. It uses XDPoS consensus, where masternodes elected through the validator contract at 0x88 produce blocks. XDPoS 2.0 is a chained HotStuff BFT protocol that uses votes, quorum certificates and a three-chain commit rule. The audit covers the fork's changes over upstream go-ethereum.

Document

Name	Blockchain Protocol Review and Security Analysis Report for Xinfin
Audited By	Tanuj Soni, Hamza Sajid
Approved By	Kerem Solmaz
Website	xinfin.org
Changelog	13/10/2026
Platform	XDC
Language	Golang
Tags	DPOS Consensus, BFT, Geth
Methodology	https://docs.hacken.io/methodologies/blockchain-protocols

Review Scope

Repository	https://github.com/XDCIndia/xdc-geth-audit
Commit	45e2b6ee2b21bb2c8aa05ef5927695c513008cc3

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

40	0	0	0
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	4
Medium	16
Low	16

Vulnerability	Severity	Status
F-2026-19525 - One Masternode Key Can Satisfy A Quorum Or Timeout Certificate	High	Pending Fix
F-2026-19601 - V2 Proposer Seal Omits Header Fields That Change The Block Hash	High	Pending Fix
F-2026-19636 - Full Sync Canonizes A Header Batch That Does Not Extend The Local Head	High	Pending Fix
F-2026-19727 - A Zero Price Signing Transaction Keeps That Account Out Of The Block	High	Pending Fix
F-2026-19457 - Destination Only Special Transactions Skip The Tip Check And The Slot Cap	Medium	Pending Fix
F-2026-19458 - A Compiled Ethstats Secret Authenticates Default Node Reporting	Medium	Pending Fix
F-2026-19460 - Verified BFT Forwarding Can Exhaust Node Through Unbounded Goroutines	Medium	Pending Fix
F-2026-19463 - Snapshot Heal Imports New V2 Headers Without Leader Turn Enforcement	Medium	Pending Fix
F-2026-19466 - Epoch Switch Full Verification Trusts Header Validators When They Are A Superset Of The Local Set	Medium	Pending Fix
F-2026-19468 - Epoch Switch Full Verification Trusts Header Fields When The V2 Snapshot Is Missing	Medium	Pending Fix
F-2026-19608 - Header Verification Accepts A Future Round With No Timeout Certificate	Medium	Pending Fix

Vulnerability	Severity	Status
F-2026-19609 - The Vote Lock Is The Parent Certificate, So A Certified Block Is Not Locked	Medium	Pending Fix
F-2026-19622 - A Vote That Completes A Certificate Latches The Next Round	Medium	Pending Fix
F-2026-19637 - Apothem Penalty Upgrade Releases A Penalized Signer After One Epoch Instead Of Five	Medium	Pending Fix
F-2026-19638 - Block Signer And Finality RPCs Read The Seal As Addresses	Medium	Pending Fix
F-2026-19639 - Epoch RPCs Use The Block Number Instead Of The V2 Round	Medium	Pending Fix
F-2026-19643 - Block Import Does Not Apply The Address Blacklist	Medium	Pending Fix
F-2026-19723 - Bulk Sync Canonizes A Block Whose State Root Does Not Match Execution	Medium	Pending Fix
F-2026-19726 - Round Timeout Uses The Round Zero Period	Medium	Pending Fix
F-2026-19773 - Chain Reorganisation Can Revert A Committed Block	Medium	Pending Fix
F-2026-19459 - Empty Transaction Dispatch Imports Unsigned TradingState Payloads Without Nonce	Low	Pending Fix
F-2026-19462 - V1 Checkpoint Validator Verification Never Rejects Populated Validators Field	Low	Pending Fix
F-2026-19467 - Block Gas Accounting Does Not Bound BlockSigners Transactions	Low	Pending Fix
F-2026-19469 - Quorum Certificate Forensics Is Never Invoked After processQC	Low	Pending Fix
F-2026-19596 - Timeout Certificates Count Penalized Signers Toward The Active Set Quorum	Low	Pending Fix
F-2026-19602 - V1 Checkpoint Extra Replaces The Signer Set And Authorizes That Sealer	Low	Pending Fix
F-2026-19612 - Timeout Gap Number Selects The Signer Set Independently Of The Round Epoch	Low	Pending Fix
F-2026-19614 - V1 Full Seal Logs The M2 Co-Sign And Does Not Reject It	Low	Pending Fix
F-2026-19621 - V1 Checkpoint Penalties Are Outside The Seal And Never Checked	Low	Pending Fix
F-2026-19640 - A Sealed Sign-Slot Block Submits A Second Signing Transaction	Low	Pending Fix
F-2026-19641 - NewBlock Imports A V1 Extension Without Recovering The Seal	Low	Pending Fix
F-2026-19642 - Bulk Sync Accepts A V2 Block Dated In The Future	Low	Pending Fix
F-2026-19644 - Apothem Protector Rewards Use Stake From The Payout Block	Low	Pending Fix
F-2026-19724 - A Non-Mainnet Insert Error Clears Bulk Sync Away From The Tip	Low	Pending Fix

Vulnerability	Severity	Status
F-2026-19774 - Voted Round, Mined Round And Lock Are Not Kept Across A Restart	Low	Pending Fix
F-2026-19815 - Epoch-Switch Verification Accepts a Reordered Validator List	Low	Pending Fix
F-2026-19471 - FastSync Auto Pin Trusts Compiled Checkpoints Without Quorum Verification	Info	Pending Fix
F-2026-19615 - V1 Header Verification Does Not Bound The Gas Limit	Info	Pending Fix
F-2026-19725 - Protector Ranking Uses A Stable Sort On Equal Stake	Info	Pending Fix
F-2026-19805 - Reward RPCs Do Not Return The Credited Epoch Reward	Info	Pending Fix

Documentation quality

- The audit relied mainly on inline code comments.
- AUDIT.md gives provenance, review order and four deliberate invariants.
- Inline comments explain many consensus rules, fork boundaries and design decisions.
- No separate functional requirements or technical specification was provided.
- The README is upstream go-ethereum's.

Code quality

- The fork keeps its changes close to upstream go-ethereum, with surgical edits and XDC logic in dedicated files.
- Consensus, hooks, sync and miner code are organized into clear packages.
- Inline comments explain many consensus rules, fork boundaries and design decisions.
- Some components are placeholders for future work, such as the DEX services and the light client backend.
- Some diagnostic logging remains in place to support parity checks with the reference client.

Architecture quality

- **Consensus:** The system implements delegated proof of stake with a HotStuff style BFT protocol, called XDPoS 2.0.
- **Scalability:** The system is a single chain. A 2 second block period, a committee of up to 108 masternodes and a high block gas limit support high throughput.
- **Decentralization:** Masternodes are elected by stake through a system contract. Fork parameters are fixed in the client, which keeps the rules consistent across nodes.
- **Interoperability:** The client is EVM compatible. It keeps a legacy peer protocol so it can work with the existing XDC network fleet.
- **Resilience:** Fallback paths exist for sync and recovery, including rollback and snapshot reseeding. Penalties exclude inactive masternodes for an epoch.

Test coverage

Go test code coverage of the XDPoS 2.0 consensus implementation is 24.8% (statement coverage). The mutation score was not measured.

- Coverage of the fork's core changes is near zero. Block production, header verification, timeout handling, epoch switching and the reward and penalty logic are largely untested.
- Vote handling, certificate processing and snapshot storage are partly covered.
- Deployment and basic user interactions are not covered by tests.
- Negative cases are not covered, for example invalid headers, forged certificates and failed rounds.
- Interactions by several masternodes are not tested thoroughly. The tests mostly use single-node and synthetic scenarios.
- The figure includes tests added during the audit. The developers' own coverage may be lower.
- Some inherited upstream tests fail because of the fork's changes.

Table of Contents

System Overview **9**

Risks **10**

Findings **11**

Vulnerability Details 11

F-2026-19525 - One Masternode Key Can Satisfy A Quorum Or Timeout Certificate - High 11

F-2026-19601 - V2 Proposer Seal Omits Header Fields That Change The Block Hash - High 14

F-2026-19636 - Full Sync Canonizes A Header Batch That Does Not Extend The Local Head - Hig h 18

F-2026-19727 - A Zero Price Signing Transaction Keeps That Account Out Of The Block - High 21

F-2026-19457 - Destination Only Special Transactions Skip The Tip Check And The Slot Cap - M edium 24

F-2026-19458 - A Compiled Ethstats Secret Authenticates Default Node Reporting - Medium 26

F-2026-19460 - Verified BFT Forwarding Can Exhaust Node Through Unbounded Goroutines - M edium 28

F-2026-19463 - Snapshot Heal Imports New V2 Headers Without Leader Turn Enforcement - Me dium 30

F-2026-19466 - Epoch Switch Full Verification Trusts Header Validators When They Are A Supers et Of The Local Set - Medium 32

F-2026-19468 - Epoch Switch Full Verification Trusts Header Fields When The V2 Snapshot Is Mi ssing - Medium 34

F-2026-19608 - Header Verification Accepts A Future Round With No Timeout Certificate - Mediu m 36

F-2026-19609 - The Vote Lock Is The Parent Certificate, So A Certified Block Is Not Locked - Me dium 38

F-2026-19622 - A Vote That Completes A Certificate Latches The Next Round - Medium 40

F-2026-19637 - Apothem Penalty Upgrade Releases A Penalized Signer After One Epoch Instead Of Five - Medium 42

F-2026-19638 - Block Signer And Finality RPCs Read The Seal As Addresses - Medium 44

F-2026-19639 - Epoch RPCs Use The Block Number Instead Of The V2 Round - Medium 46

F-2026-19643 - Block Import Does Not Apply The Address Blacklist - Medium 48

F-2026-19723 - Bulk Sync Canonizes A Block Whose State Root Does Not Match Execution - Me dium 50

F-2026-19726 - Round Timeout Uses The Round Zero Period - Medium 52

F-2026-19773 - Chain Reorganisation Can Revert A Committed Block - Medium 2

F-2026-19459 - Empty Transaction Dispatch Imports Unsigned TradingState Payloads Without No nce - Low 56

F-2026-19462 - V1 Checkpoint Validator Verification Never Rejects Populated Validators Field - L ow 58

F-2026-19467 - Block Gas Accounting Does Not Bound BlockSigners Transactions - Low 60

F-2026-19469 - Quorum Certificate Forensics Is Never Invoked After ProcessQC - Low 62

F-2026-19596 - Timeout Certificates Count Penalized Signers Toward The Active Set Quorum - Low 64
F-2026-19602 - V1 Checkpoint Extra Replaces The Signer Set And Authorizes That Sealer - Low 66
F-2026-19612 - Timeout Gap Number Selects The Signer Set Independently Of The Round Epoch - Low 68
F-2026-19614 - V1 Full Seal Logs The M2 Co-Sign And Does Not Reject It - Low 71
F-2026-19621 - V1 Checkpoint Penalties Are Outside The Seal And Never Checked - Low 73
F-2026-19640 - A Sealed Sign-Slot Block Submits A Second Signing Transaction - Low 75
F-2026-19641 - NewBlock Imports A V1 Extension Without Recovering The Seal - Low 77
F-2026-19642 - Bulk Sync Accepts A V2 Block Dated In The Future - Low 79
F-2026-19644 - Apothem Protector Rewards Use Stake From The Payout Block - Low 81
F-2026-19724 - A Non-Mainnet Insert Error Clears Bulk Sync Away From The Tip - Low 83
F-2026-19774 - Voted Round, Mined Round And Lock Are Not Kept Across A Restart - Low 85
F-2026-19815 - Epoch-Switch Verification Accepts A Reordered Validator List - Low 88
F-2026-19471 - FastSync Auto Pin Trusts Compiled Checkpoints Without Quorum Verification - Info 90
F-2026-19615 - V1 Header Verification Does Not Bound The Gas Limit - Info 93
F-2026-19725 - Protector Ranking Uses A Stable Sort On Equal Stake - Info 95
F-2026-19805 - Reward RPCs Do Not Return The Credited Epoch Reward - Info 97
Observation Details 99
Disclaimers 100
Hacken Disclaimer 100
Technical Disclaimer 100
Appendix 1. Severity Definitions 101
Appendix 2. Scope 102
Components In Scope 102
Appendix 3. Additional Valuables 104
Frameworks And Methodologies 104

System Overview

XDC is the execution client, a fork of go-ethereum v1.17.5 serving XDC mainnet and the Apothem testnet. It implements XDPoS, a delegated proof of stake consensus in which masternodes elected through a system contract produce and finalize blocks. Version 2 of the protocol, XDPoS 2.0, is a HotStuff style BFT design. The audit covers the fork's changes over upstream go-ethereum.

- **Validator election**. The validator contract at `0x88` holds the candidates and their stake. An epoch lasts 900 blocks, and the committee has up to 108 masternodes on mainnet. At the gap block, 450 blocks before the epoch switch, the client reads the candidates, orders them by stake, and stores the result as a snapshot. At the epoch switch, the penalty hook builds the penalty list, and the proposer writes the next masternode set and that list into the block header. In normal operation, every node recomputes and checks both lists.
- **Rounds and leaders**. Time is divided into rounds. Each round has one leader, chosen from the masternode list by the round number. The leader builds a block that carries the highest quorum certificate it knows. That certificate covers an earlier block: the parent in an ordinary round, or an older ancestor when the chain has moved forward through timeouts. The block is sealed with the leader's signature in the header.
- **Voting and certificates**. Masternodes execute the block, check the state root, and then vote. A vote has to extend the locked certificate. When votes from two thirds of the masternode set are collected, they form a quorum certificate. If a round fails, timeout messages form a timeout certificate and the next leader continues. Sync messages carry the latest certificates to nodes that fall behind.
- **Finality.** Three linked blocks in consecutive rounds, with the newest certified, finalize the oldest block. A commit hook then moves the chain's finalized marker.
- **Hooks and system contracts.** Reward and penalty hooks connect the engine to the system contracts. The commit hook moves the finalized marker. The consensus path reads the candidates, their stake, and the reward owner directly from the validator contract's storage. Transactions to the validator contract and to the randomize contract still run in the EVM. Masternodes send a gasless signing transaction to the block signer contract at `0x89` on every block whose number is divisible by 15. After the signing fork, that transaction is recorded as a zero-gas receipt. It records attendance for the penalty comeback check and, on Apothem, for protector rewards. Rewards are paid at the epoch switch: 90% to the masternode owner and 10% to the foundation wallet.
- **Core integration.** The fork adds the next validator set, the proposer seal, and the penalty list to the block header. It also adds special handling for the gasless transactions, a fixed base fee, and its own fork schedule. It supports account balances above the normal limit. The transaction pool, state storage, and chain database were extended to carry these changes.
- **Networking and sync**. Nodes keep a legacy peer protocol so they interoperate with the existing XDC fleet. Votes, timeouts, and sync messages are carried on this protocol and routed to the consensus engine. A custom sync loop and a restored fast sync mode download state from trusted checkpoints, with fallbacks for gaps and healing.
- **Block production.** A dedicated worker loop decides when the node is the leader, builds the block, seals it, inserts it, and broadcasts it. On a signing slot it broadcasts the signing transaction as well. The block gas limit is held at a fixed value.

Risks

- **Reliance on system contracts outside the audit scope**: Masternode election and rewards depend on system contracts whose sources and deployed code are outside this audit. The client trusts the data held by those contracts, so the accuracy of that data cannot be fully verified here.
- **No economic penalty for misbehavior:** The only sanction is exclusion from the committee for one epoch. Signing conflicting blocks carries no penalty, and no stake is slashed. The cost of attacking safety is therefore low relative to the stake.
- **Trust anchors during sync**: Fresh nodes can start from compiled or operator-supplied checkpoints and may take their validator set from other nodes. While a node is catching up, verification is relaxed. The validator set it uses therefore depends on these anchors.

Findings

Vulnerability Details

F-2026-19525 - One Masternode Key Can Satisfy A Quorum Or Timeout Certificate - High

Description:

XDPoS 2.0 treats a quorum certificate as proof that `CertThreshold` of the seated masternodes (about two thirds) signed the same vote. The check that enforces that proof counts distinct signature byte strings. It does not count distinct recovered addresses.

`verifyQC` drops only byte-identical copies, then compares the remaining length with `MasternodesLen * CertThreshold`. Each surviving signature is passed to `verifyMsgSignature`, which returns success when the recovered address is any member of the masternode list. The recovered address is discarded, so the same address can satisfy the check once per signature.

```
signatures, duplicates := UniqueSignatures(quorumCert.Signatures)
// ...
if qcRound > 0 && (signatures == nil || float64(len(signatures)) <
float64(epochInfo.MasternodesLen)*certThreshold) {
return utils.ErrInvalidQCSignatures
}
verified, _, err := x.verifyMsgSignature(signedVoteObj, signature,
epochInfo.Masternodes)
```

`UniqueSignatures` keys its map with `common.BytesToHash(sig)`. `verifyTC` uses the same helper and the same length check against `CertThreshold`. `RecoverUniqueSigners` in the same file already groups signatures by recovered address, and it is not called on either path.

ECDSA allows one private key to produce many verifying signatures over one hash by signing with a different nonce each time. `crypto.Ecrecover` accepts each of them and returns the same address. One seated masternode can therefore assemble a certificate whose signature count meets the threshold while every signature recovers to that single key.

The vote pool and the timeout pool admit the same set. `Pool.Add` dedupes by `Hash()`, and `Vote.Hash` / `Timeout.Hash` cover the signature bytes, so two signatures from one signer are two pool entries. `onVotePoolThresholdReached` appends every pooled vote whose signer field is non-empty and calls `processQC` with that list. `processQC` updates `highestQuorumCert`, runs the 3-chain commit, and calls `setNewRound` without a second pass over signer identity. `onTimeoutPoolThresholdReached` copies every pooled timeout signature into a `TimeoutCert` and calls `processTC`.

Header verification accepts this quorum certificate through `verifyQC`. A timeout certificate is accepted in sync info through `verifyTC`, and the timeout pool passes the same signatures to `processTC`.

One masternode can finalize a block, open the next round, or force a timeout round without the rest of the committee signing.

Assets:

- XDPoS consensus [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 5/5

Likelihood Rate: 3/5

Severity: High

Recommendations

Remediation:

- Before the `CertThreshold` comparison in `verifyQC` and `verifyTC`, keep one signature per recovered masternode address. Compare that count with `MasternodesLen * CertThreshold`.
- Apply the same address set in `onVotePoolThresholdReached` and `onTimeoutPoolThresholdReached` before `processQC` and `processTC`.
- Key the vote pool and the timeout pool by signer address inside each pool key, so one masternode contributes one message per round and block.

Evidences

POC for One Signer with multiple nonces bypassing network quorum

Reproduce:

1. Certificate accepted (`TestSingleSignerQuorum_OneOfTenKeysFillsTheCertificate`)
    1. Seat 10 distinct masternode keys. `CertThreshold` is 2/3, so the gate is `10 * 2/3 = 6.666…`. A count of 7 passes and a count of 6 fails.
    2. Build one `VoteForSign` for one block and one gap. Key 0 signs that hash 7 times with `ecdsa.Sign` and a new nonce each time. Keys 1 through 9 do not sign.
    3. `Ecrecover` on all 7 signatures returns key 0. `UniqueSignatures` keeps all 7 and reports 0 duplicates.

4. `verifyQC` on those 7 signatures returns nil.
5. `verifyQC` on the first 6 of the same signatures returns an error.

2. Block finalized (`TestSingleSignerQuorum_OneOfTenKeysFinalizesViaVotePool`)
   1. The chain is three consecutive headers: block 5850 round 4050, block 5851 round 4051, block 5852 round 4052. The votes name block 5852. The engine round is 4052.
   2. Deliver the 7 votes, one at a time, through `VoteHandler`.
   3. After votes 1 through 6, `highestCommitBlock` is empty and `currentRound` is still 4052.
   4. Vote 7 crosses the pool threshold. `onVotePoolThresholdReached` builds a quorum certificate from those signatures and calls `processQC`.
   5. `HookCommitBlock` is called with block 5850. `highestCommitBlock` is block 5850 at round 4050. That is the grandparent, which is the block the three-chain rule finalizes when the tip certificate is accepted.
   6. `currentRound` becomes 4053.
   7. `highestQuorumCert` certifies block 5852 and holds 7 signatures. Each one recovers to key 0.

Files: single_signer_quorum_test.go

F-2026-19601 - V2 Proposer Seal Omits Header Fields That Change The Block Hash - High

Description:

V2 header verification recovers the proposer from `header.Validator` over `sigHash`. That hash covers the header through `BaseFee`. It does not cover `WithdrawalsHash`, `BlobGasUsed`, `ExcessBlobGas`, `ParentBeaconRoot`, or `RequestsHash`. The block hash is the keccak256 of the header RLP, and that RLP includes those fields.

A peer relaying the sealed block can set `WithdrawalsHash` to a 32-byte hash without the proposer's key. The signature bytes in `header.Validator` stay as the proposer produced them, and the quorum certificate in `Extra` stays inside `sigHash`. `ecrecover` still returns the leader for that round, because it hashes `sigHash` and not `Header.Hash`.

The header RLP writes each optional field when that field or any later optional field is set. After the XDPoS fields, that tail is `BaseFee`, `WithdrawalsHash`, `BlobGasUsed`, `ExcessBlobGas`, `ParentBeaconRoot`, and `RequestsHash`. A nil hash in the tail is written as an empty string, and the decoder requires 32 bytes for a hash. A header that sets `ParentBeaconRoot` or `ExcessBlobGas` while `WithdrawalsHash` is nil therefore does not decode, and it never reaches `verifyHeader`.

A header that sets `WithdrawalsHash` to the empty-withdrawals root does decode. `sigHash` is unchanged and `Header.Hash` changes. `ValidateBody` accepts the block when the body carries an empty withdrawals list, because that list hashes to the same root. `ShanghaiTime` is unset on XDC mainnet and Apothem, so execution does not apply withdrawals and the state root stays the same. `verifyHeader` does not require `WithdrawalsHash` to be absent. Clique header verification rejects a non-nil `WithdrawalsHash`. The V1 seal encoder appends `WithdrawalsHash` when it is set. The V2 path does neither.

`NewBlock` imports a block that extends the chain head, and it also imports a same-height sibling whose parent is the head's parent. After `InsertChain` returns, it calls `HandleProposedBlock`. The vote names `header.Hash()`, and the vote-pool key includes that hash. A node that has not yet voted in the round votes for the hash it imported. Votes on the two hashes do not add, so the round does not form one quorum certificate.

The same peer can also set `ExcessBlobGas` once `WithdrawalsHash` is a 32-byte hash, which is what allows the header to decode. That header passes `verifyHeader`, because `verifyHeader` does not read `ExcessBlobGas`. Import then calls `Process`. `Process` calls `NewEVMBlockContext`, and `NewEVMBlockContext` calls `CalcBlobFee` whenever `ExcessBlobGas` is set. Mainnet and Apothem leave

`BlobScheduleConfig` unset, and Cancun is not a timestamp fork, so `CalcBlobFee` panics. The panic is inside `Process`, before `writeBlockAndSetHead`, so this import does not reach `collectReceiptsAndLogs`. `collectReceiptsAndLogs` calls `CalcBlobFee` only for a block that is already stored.

Honest nodes share `sigHash` and `verifyHeader`, so the original header and the header with `WithdrawalsHash` set both verify under the same proposer signature.

The seal hash stops at `BaseFee`:

```
enc := []interface{}{
header.ParentHash,
header.UncleHash,
header.Coinbase,
header.Root,
header.TxHash,
header.ReceiptHash,
header.Bloom,
header.Difficulty,
header.Number,
header.GasLimit,
header.GasUsed,
header.Time,
header.Extra,
header.MixDigest,
header.Nonce,
header.Validators,
header.Penalties,
}
if header.BaseFee != nil {
enc = append(enc, header.BaseFee)
}
```

The condition that must hold is a V2 header whose proposer signature recovers to the leader, with `WithdrawalsHash` set to a 32-byte hash the body matches. A relaying peer can produce that header without the proposer key. Nodes that import it vote for a different block hash than nodes that import the header the proposer sealed.

Assets:

- XDPoS consensus [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 4/5

Likelihood Rate: 4/5

Severity: High

Recommendations

Remediation:

- Include in `sigHash` every field that `Header.EncodeRLP` writes, except the signature bytes in `header.Validator`.
- In `verifyHeader`, reject a non-nil `WithdrawalsHash`, `BlobGasUsed`, `ExcessBlobGas`, `ParentBeaconRoot`, or `RequestsHash` while Shanghai, Cancun, and Prague are inactive. That rejection does not replace the seal coverage.

Evidences

V2 proposer seal omits header fields that change the block hash.

Reproduce:

Create a poc directory in the root of the XDPoS source tree, next to go.mod. Place the downloaded main.go in that directory. From the source tree root, run:

```
go run ./poc
```

The program builds and checks the following:

1. A V2 header at block 80,370,001 with `BaseFee` set and the optional tail unset. `SignHash`, which is the V2 `sigHash`, is signed with a generated key.
2. A copy with `ParentBeaconRoot` set and `WithdrawalsHash` nil. RLP encode then decode fails, because the encoder wrote `WithdrawalsHash` as an empty string.
3. A copy of the original header with `WithdrawalsHash` set to the empty-withdrawals root. Encode and decode succeed.
4. `SignHash` on the decoded header equals the original `sigHash`, and `Header.Hash` differs from the original block hash.
5. Recovery of the original signature over the altered digest returns the signer of the original header.

Results:

```
ParentBeaconRoot with nil WithdrawalsHash: encode 547 bytes, decode
error: rlp: input string too short for common.Hash, decoding into (
types.Header).WithdrawalsHash
WithdrawalsHash set: decode ok
sigHash unchanged: true
block hash changed: true
same signature recovers signer on both: true
original block hash: 0x8e6500fd9eb2a6eb116910d16736029e8e7f0cd07a8b
d83d4642c74908309fe4
altered block hash: 0xc4b8531890efa91f3bf0cb3fbfc256980dd27f4963c75
e4db252c302bfac861d
```

`SignHash` is the V2 seal. Setting `WithdrawalsHash` to the empty-withdrawals root leaves that digest unchanged and changes `Header.Hash`. The same signature recovers on both headers.
Setting `ParentBeaconRoot` while `WithdrawalsHash` is nil does not decode.

Files:	main.go
	README.md

F-2026-19636 - Full Sync Canonizes A Header Batch That Does Not Extend The Local Head - High

Description:

A default node starts `xdcSyncer` from `handler.Start` with `XdcBulkSyncMode` set. Experimental snap sync is unset, so the phase stays on full synchronization and `syncOnce` calls `fullSyncOnce`. Each round requests headers beginning at the chain head plus one.

A peer that answers that request supplies the batch. `handleBlockHeaders` delivers it on `peer.XDCHeaderResp` before locating the requested anchor. The dispatcher then keeps the suffix that starts at that anchor. `fetchHeaderBatchTimeout` returns the list from `XDCHeaderResp` as the peer sent it. `fullSyncOnce` and the lookahead import build blocks from that list and call `InsertChain`. Neither compares `headers[0].Number` with the block it requested. Ancestor prefetch and the forward fetch both return when the first header is not that block. The lookahead import records `headers[0].Number` as the range it imported.

`InsertChain` runs with the head-update flag set. When the parent hash is not the chain head, `writeBlockAndSetHead` reorganizes the chain and stores the inserted block as the head. Total difficulty is not compared. A long reorganization is logged and is still committed.

While `XdcBulkSyncMode` is set, V1 headers go through `verifyHeaderLightV1`. That check requires a stored parent and the V1 header shape, and it does not recover the seal. Mainnet and Apothem stay on this path until the V2 switch. After the switch, the peer must supply a header with a quorum certificate and a 65-byte proposer signature that recovers to the leader for the round. The same insert still updates the head. The proof covers the V1 path only.

A canonical prefix leaves the head in place, and a missing parent returns `ErrUnknownAncestor`. Every node that runs `fullSyncOnce` imports the peer's batch the same way.

The handler delivers the peer's headers before the anchor is located:

```
select {
case peer.XDCHeaderResp <- headers:
default:
}
```

`fullSyncOnce` inserts that list:

```
n, err := s.handler.chain.InsertChain(blocks)
```

The head write follows a parent that is not the chain head:

```
if block.ParentHash() != currentBlock.Hash() {
if err := bc.reorg(currentBlock, block.Header()); err != nil {
return NonStatTy, err
}
}
bc.writeHeadBlock(block)
```

The condition that must hold is a full-synchronization import of a peer's batch whose first new block has a stored parent, a parent hash other than the chain head, and a header that passes the verification active for that height. When it holds, that block becomes the canonical head.

Assets:

- Mining / sync [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 4/5

Likelihood Rate: 3/5

Severity: High

Recommendations

Remediation:

- Before `InsertChain` in `fullSyncOnce` and in the lookahead import, require `headers[0].Number` to be the requested block and `headers[0].ParentHash` to be the current head.
- Reject the batch otherwise, as ancestor prefetch and the forward fetch already reject a response that starts at the wrong block.
- Pass `fullSyncOnce` the header list after the anchor has been located, rather than the list delivered on `XDCHeaderResp` before that step.

Evidences

Full sync canonizes a header batch that does not extend the local head.

Reproduce: Create a `poc` directory in the root of the XDPoS source tree, next to `go.mod`. Place the downloaded `main.go` in that directory. From the source tree root, run:

```
go run ./poc
```

The program builds and checks the following:

1. An XDPoS chain with `XdcBulkSyncMode` set, the mode `xdcSyncer` starts in, and eight V1 blocks. Each block carries a 32-byte vanity and a 65-byte seal that is not a signature. After insert, the chain head is block 8, so the next full-sync request is block 9.
2. The same eight-block batch inserted again. The head is unchanged, because the batch is a prefix of the canonical chain.
3. One block whose parent hash is not stored. The import returns unknown ancestor, and the head is unchanged.
4. One block on canonical block 2, so the delivered header is block 3, with `Validator` set to 65 bytes that do not recover. `Author` returns an invalid signature recovery id. Block 3 is not the requested block 9, its parent hash is not the chain head, and its cumulative difficulty is below the canonical chain.
5. That block inserted through `InsertChain`, the call `fullSyncOnce` and the lookahead import make. The inserted block becomes the canonical head, and block 8 is no longer canonical at its number.
6. The V1 light path only. `verifyHeaderLightV1` does not recover the seal. After the V2 switch, a peer must supply a header that contains a quorum certificate and a 65-byte proposer signature that recovers to the leader for the round. This program does not build that header. The same `InsertChain` still stores a block that passes the active verification as the head.

Results:

```
canonical head: 8 0x9f842c3996ccabfd37d772a1965987eb53b2ab8eb238f6c
1474c00687b4dd408
canonical cumulative difficulty: 9
prefix of the canonical chain, head unchanged: true
missing parent: unknown ancestor
head unchanged after missing parent: true
seal recovery: invalid signature recovery id
full sync requested block: 9
delivered headers[0].Number: 3
headers[0].Number is the requested block: false
headers[0].ParentHash is the current head: false
fork cumulative difficulty: 4
head after InsertChain: 3 0xbc9deeca33b89571da690a462f96004b1048b31
15a61eccc92cc908480e0e45d
head moved to the delivered block: true
previous head is still canonical at its number: false
```

`XdcBulkSyncMode` is set, which is the mode `xdcSyncer` starts in. The chain is at block 8, so the next full-sync request is block 9. The delivered block is block 3. Its parent is stored, its parent hash is not the local head, and its cumulative difficulty is 4 against the canonical chain's 9. The extra seal does not recover. `InsertChain` still makes that block the canonical head, and block 8 is no longer canonical. A prefix of the canonical chain leaves the head in place. A block with a missing parent returns `unknown ancestor` and leaves the head in place.

Files: main.go

F-2026-19727 - A Zero Price Signing Transaction Keeps That Account Out Of The Block - High

Description:

`Pending` keeps a transaction to block signers (`0x89`) or randomize (`0x90`) when its tip is below the miner minimum. The comment on that exemption says the miner must include it. `CreateTxSign` builds the signing transaction with gas price 0, and `broadcastSigningTx` submits it from the coinbase on every block number divisible by 15, when the block is near the head and a wallet is available. The randomize builders use the same gas price of 0. Past the fee-market fork the block base fee is `XDCBaseFee`, 12.5 gwei. A legacy transaction's fee cap is its gas price, so the cap is 0. Mainnet activates that fork at block 98,800,200 and Apothem at block 71,550,000. Both networks are past those blocks.

`fillTransactions` passes the header base fee into `NewTransactionsByPriceAndNonce`. The ordering wraps each account's first transaction with `newTxWithMinerFee`. That function returns `ErrGasFeeCapTooLow` when the fee cap is below the base fee. It does not test `IsSpecialTransaction`. When the failing transaction is the account's next nonce, the constructor removes the account, so later nonces are not ordered and a higher fee from the same account is not compared. `commitTransactions` only reads that ordering. When the zero-price transaction sits behind a nonce that was accepted, `Shift` uses the same check, removes the account from the ordering, and does not advance to the following nonce.

`applyTransaction` would run a selected `0x89` transaction through `ApplySigningTransaction` once the signing fork is active. The ordering never selects it. A transaction to `0x90` fails the same fee-cap check before execution. Every sealer uses this ordering. The node that submitted the signing transaction has it pending, so the block it seals omits that account. A peer that has the same transaction omits it as well.

The pool keeps the special transaction:

```
if !tx.IsSpecialTransaction() && tx.EffectiveGasTipIntCmp(filter.Mi
nTip, filter.BaseFee) < 0 {
txs = txs[:i]
break
}
```

The ordering removes the account:

```
if baseFee != nil {
if tx.GasFeeCap.Cmp(baseFee) < 0 {
return nil, types.ErrGasFeeCapTooLow
}
}
```

```
wrapped, err := newTxWithMinerFee(accTxs[0], from, baseFeeUint)
if err != nil {
delete(txs, from)
continue
}
```

The condition that must hold is a pending transaction to `0x89` or `0x90` whose gas price is 0, at the account's next nonce, in a block whose base fee is set. The sealed block then omits that transaction and every later nonce from the same account. Masternodes submit every signing transaction at gas price 0, so blocks sealed by this client contain no signing transactions, and masternodes lose the signing credit that the epoch reward is split by.

Assets:

- Mining / sync [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 3/5

Likelihood Rate: 5/5

Severity: High

Recommendations

Remediation:

- In `newTxWithMinerFee`, treat `IsSpecialTransaction` as a zero tip and return the wrapped transaction.
- Keep the fee-cap check for every other transaction.

Evidences

A zero-price signing transaction keeps that account out of the block

Reproduce: Create a `poc` directory in the root of the XDPoS source tree, next to `go.mod`. Place the downloaded `main.go` in that directory. From the source tree root, run:

```
go run ./poc
```

Results:

```
signing destination: 0x0000000000000000000000000000000000000089
signing gas price: 0
signing fee cap: 0
```

```
base fee: 12500000000
special transaction: true
fee cap below base fee: fee cap less than base fee
pool would keep it: true
next nonce is the signing transaction
selected sender is the other account: true
signing transaction selected: false
later nonce from the same account selected: false
payable nonce selected before the signing transaction: true
signing transaction selected after that nonce: false
randomize destination: 0x0000000000000000000000000000000000000090
randomize gas price: 0
randomize account selected: false
```

`CreateTxSign` sends the signing transaction to `0x89` with gas price 0. The fee cap is that gas price, and the block base fee is 12.5 gwei, so the cap is below the base fee. The transaction is a special transaction, so the pool keeps it. `NewTransactionsByPriceAndNonce` does not select it. A higher-priced transaction at the next nonce of the same account is not selected either. A different account is selected. When the signing transaction sits behind a payable nonce, that nonce is selected and the signing transaction is not. A zero-price transaction to `0x90` is removed the same way.

Files: main.go

F-2026-19457 - Destination Only Special Transactions Skip The Tip Check And The Slot Cap - Medium

Description:

`IsSpecialTx` is true when the destination is `BlockSignersBinary` (`0x89`) or `RandomizeSMCBinary` (`0x90`). The pool does not require a masternode sender, a `Sign` selector, or a non-zero gas price.

Validation still recovers the sender and checks the nonce. `tx.Cost` is value plus gas limit times gas price. A transaction with gas price 0 and value 0 has cost 0, so the balance check does not reject an empty account. The per-account slot check still runs. The minimum-tip check does not, and the global slot check does not. A matching destination is admitted while `all.Slots()` is already above `GlobalSlots + GlobalQueue`.

The transaction is then stored in the priced list. A later non-special transaction, once the pool is over that cap, calls `priced.Discard` and drops the cheapest entries. Those entries are the zero-price special transactions, so the fee-paying transaction is the one kept. Raising the pool tip calls `TxsBelowTip`, which drops every stored transaction under the new tip, including these. `truncatePending` on a head reset drops accounts whose pending count is above `AccountSlots`. An account at or below `AccountSlots` is kept. Many accounts, each at that allowance, can remain above `GlobalSlots` after the reset.

`Pending` keeps the same destinations when the miner filter has a non-zero minimum tip, because that comparison exempts `IsSpecialTransaction`. After `TIPSigning`, import of a `BlockSigners` transaction uses `ApplySigningTransaction`. That path checks the signature and the nonce. It does not check that the sender is a masternode, and it does not debit block gas. Reward credit uses `isSigningTx`, which requires destination `0x89`, calldata length 68, and the `Sign` selector. Other calldata does not receive that credit. A `Randomize` destination is still an ordinary EVM call. It is admitted with no tip, and inclusion still consumes block gas.

Any account that can sign to `0x89` or `0x90` can occupy slots above the global cap. The per-account allowance still applies. No masternode key is required.

Special status is the destination alone:

```
func IsSpecialTx(to *common.Address) bool {
return to != nil && (*to == common.BlockSignersBinary || *to == com
mon.RandomizeSMCBinary)
}
```

The minimum-tip check skips that destination:

```
if !tx.IsSpecialTransaction() && tx.GasTipCapIntCmp(opts.MinTip) <
0 {
return fmt.Errorf("%w: gas tip cap %v, minimum needed %v", ErrTxGas
PriceTooLow, tx.GasTipCap(), opts.MinTip)
}
```

The global slot check skips the same destination:

```
if uint64(pool.all.Slots()+numSlots(tx)) > pool.config.GlobalSlots+
pool.config.GlobalQueue &&
!tx.IsSpecialTransaction() {
```

`truncatePending` only queues accounts above the per-account allowance for removal:

```
if length > pool.config.AccountSlots {
spammers.Push(addr, length)
}
if pending <= pool.config.GlobalSlots {
return
}
```

Assets:

- Core [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 2/5

Likelihood Rate: 3/5

Severity: Medium

Recommendations

Remediation:

- Treat a transaction as special only when the destination is `BlockSignersBinary` or `RandomizeSMCBinary`, the sender is a seated masternode, and, for `BlockSignersBinary`, the calldata is the `Sign` selector.
- Apply the minimum-tip check and the `GlobalSlots + GlobalQueue` check to every other transaction to those addresses.
- Keep the per-account slot check. It already runs for these destinations.
- Do not rely on `truncatePending` to restore `GlobalSlots`. It leaves accounts that are at or below `AccountSlots`.

F-2026-19458 - A Compiled Ethstats Secret Authenticates Default Node Reporting - Medium

Description:

The ethstats host and secret are stored in the client binary as package constants. Every XDPoS node therefore loads the same credential. Registration runs when the `ethstats` flag is absent. The connection string is assembled from that host, that secret, and the public address of the machine, resolved through a third-party geolocation service.

A caller who possesses the constant can present an internally consistent node name that includes the chain identifier, the sync mode, and an IP of their choosing. Local checks do not prevent impersonation: the board authenticates the shared secret, and the reported name is taken from the client.

Because honest operators share the same compiled credential, they connect to the same board under the same secret. Presence on the board then confirms possession of a public constant rather than independently issued operator credentials. Changing the board secret requires a rebuild of every binary that embeds the constant.

An XDPoS chain configuration is still required. The secret is not a sealing key and does not admit the holder to the masternode set.

Every XDPoS binary embeds the board host and secret:

```
xdcDefaultStatsHost = "stats.xdcindia.com:443"
xdcDefaultStatsSecret = "xdc_openscan_stats_2026"
```

The XDPoS backend always registers the service:

```
if eth != nil && eth.BlockChain().Config().XDPoS != nil {
utils.RegisterEthStatsService(stack, backend, utils.XDCEthstatsURL(
cfg.Ethstats.URL, chainID, nodetype))
}
```

The synthesized URL authenticates with the compiled secret:

```
return name + ":" + xdcDefaultStatsSecret + "@" + xdcDefaultStatsHo
st
```

The condition requires an XDPoS chain and a reachable statistics board. Default registration uses the compiled secret when `--ethstats` is absent. The reported name is chosen by the client. `XDCEthstatsURL` queries `ipinfo.io` for every XDPoS node, including when `--ethstats` is set.

A holder of the constants can produce the spoofed board entries and can read the default telemetry stream, limited only by whatever access control

the board applies after the shared password. Consensus voting power is unaffected.

Assets:

- CMD [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 2/5

Likelihood Rate: 4/5

Severity: Medium

Recommendations

Remediation:

- Register the statistics client only when `--ethstats` is set. Leave `XDCEthstatsURL` unused on the default XDPoS path.
- Take the host and the secret from that flag. Do not embed them as `xdcDefaultStatsHost` and `xdcDefaultStatsSecret`.
- Call `xdcPublicInfo` only when the operator opts in. Do not query `ipinfo.io` on every XDPoS start.
- Rotate the board password `xdc_openscan_stats_2026`. Issue a separate credential to each operator.

F-2026-19460 - Verified BFT Forwarding Can Exhaust Node Through Unbounded Goroutines - Medium

Description:

Inbound XDPoS V2 votes, timeouts, and sync-info messages are checked before they are forwarded. A message that fails verification returns and does not start a goroutine. A message that verifies is sent on an unbuffered channel from a new goroutine, and then passed to the consensus handler.

`New` creates that channel with no buffer. `SetBroadcastCh` replaces it with the engine's `BroadcastCh`, which is also unbuffered. `broadcastToBftChannel` sends the engine's own votes, timeouts, and sync info on that same channel, again from a new goroutine. One receiver, `Bfter.loop`, reads it.

While the loop is running, each channel send completes when the loop receives the value. The loop then starts another goroutine. For a vote that goroutine is `Broadcast.Vote`, which calls `Peer.Send` for every connected peer. `Broadcast.Timeout` and `Broadcast.SyncInfo` do the same. Nothing limits how many of those goroutines are in flight, and nothing drops a message when a peer send is slow.

`Stop` closes `quit` and does not close the channel. After the loop returns, a later verified send blocks on the channel for the life of the process.

Votes and sync info are discarded when the block number is more than 7 away from the chain head. Timeouts are discarded when the gap is more than three epochs from the head. Those checks do not count goroutines.

A verified signature is still required for an inbound message. The engine's own output does not pass through that check. This path does not skip quorum verification of a block.

A verified vote is sent, then handled:

```
go func() { b.broadcastCh <- vote }()
if err := b.engine.VoteHandler(b.chain, vote); err != nil {
log.Debug("handle BFT Vote", "error", err)
return err
}
```

The loop starts one broadcast goroutine per object:

```
case *types.Vote:
go b.broadcast.Vote(v)
case *types.Timeout:
go b.broadcast.Timeout(v)
case *types.SyncInfo:
go b.broadcast.SyncInfo(v)
```

`Stop` closes only the quit signal:

```
func (b *Bfter) Stop() {
close(b.quit)
}
```

Assets:

- Core [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 2/5

Likelihood Rate: 3/5

Severity: Medium

Recommendations

Remediation:

- Keep the number of outstanding goroutines for verified votes, timeouts, and sync-info messages within a fixed capacity, including the goroutine that broadcasts each message to peers.
- After the forwarding loop has stopped, a later verified message must not block for the life of the process.
- A verified BFT signature must not be the only limit on how many of these goroutines are outstanding.

F-2026-19463 - Snapshot Heal Imports New V2 Headers Without Leader Turn Enforcement - Medium

Description:

V2 header verification has a full mode and a reduced mode. Full mode returns `ErrNotItsTurn` when the recovered proposer is not the masternode selected for the round. Reduced mode logs that mismatch and continues. `xdcFullVerify` selects the mode from two process-wide flags. Either flag turns full mode off for every V2 header in the process.

Fast-sync and snapshot restore leave `CheckpointSyncNoStateActive` set when the head state is missing. Reaching the advertised head clears `XdcBulkSyncMode`. `ClearCheckpointSyncNoState` then refuses to clear the remaining flag, because `HasState` is false for that head. Snap heal clears the flag only when `XDC_SNAP_HEAL` is set and the healed root is present. Until one of those clears succeeds, new tip headers are still verified in reduced mode.

`verifyQC` still runs. The proposer must still recover to an address in the masternode list, and that address must match the coinbase. The check that does not run is leader turn. A non-leader in that list can propose a header whose quorum certificate verifies. The hash is then stored in `verifiedHeaders`. The cache records the hash only. A later call for the same hash returns before the leader check, including after full mode resumes.

Reduced mode is the path the engine uses while `XdcBulkSyncMode` is set, including for ancestors that are still inside the import batch. That flag is already false in the window above. The window is the interval after the local head matches a peer head, while `CheckpointSyncNoStateActive` is still set.

Full verification is off for the whole process while either flag is set:

```
func xdcFullVerify() bool {
return !core.XdcBulkSyncMode.Load() && !core.CheckpointSyncNoStateA
ctive.Load()
}
```

Leader turn logs and continues unless full verification is on:

```
if masterNodes[leaderIndex] != validatorAddress {
if !fullVerify {
log.Warn("[verifyHeader] A.87 leader mismatch tolerated during bulk
sync",
"block", header.Number, "curIndex", curIndex, "leaderIndex", leader
Index,
"expected", masterNodes[leaderIndex], "actual", validatorAddress)
} else {
return utils.ErrNotItsTurn
}
}
```

A passing verification is cached by hash:

```
_, ok := x.verifiedHeaders.Get(header.Hash())
if ok {
return nil
}
```

A non-leader block with a valid quorum certificate can be imported during that window and kept through the cache. Forming the certificate still requires a supermajority of vote signatures.

Assets:

- XDPoS consensus [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Severity: Medium

Recommendations

Remediation:

- In `verifyHeader`, return `ErrNotItsTurn` for a header at the current head when `CheckpointSyncNoStateActive` is set.
- Keep the reduced leader check for in-batch ancestors while `XdcBulkSyncMode` is set.
- Record in `verifiedHeaders` whether the entry was produced with `fullVerify` false, and run the leader check again when a later call has `fullVerify` true.
- Leave `ClearCheckpointSyncNoState` refusing to clear the flag when `HasState` is false.

F-2026-19466 - Epoch Switch Full Verification Trusts Header Validators When They Are A Superset Of The Local Set - Medium

Description:

An epoch-switch header carries a `Validators` field. `verifyHeader` compares that field with the masternode list from `calcMasternodes`. When the lists differ and the header list is longer, `isMasternodeSubset` checks that every local address appears in the header list. If that check passes, verification logs a warning and replaces the local list with the header field. The replacement runs when `fullVerify` is true. `HookPenalty` being set and the snapshot being present do not stop it.

The same assignment copies `header.Penalties` onto the local penalty list. The penalty comparison that follows then compares the header field with itself.

The quorum certificate in the header is verified before this comparison. After the replacement, the proposer must recover to an address in the header list, and that address must be the leader for the round in that longer list. `GetMasternodesFromEpochSwitchHeader` later reads masternodes from the imported header field.

Epoch-switch verification compares the two lists, then takes the longer header list:

```
validatorsAddress := contracts.ExtractAddressFromBytes(header.Valid
ators)
if !utils.CompareSignersLists(masterNodes, validatorsAddress) {
isSupersetCase := len(masterNodes) < len(validatorsAddress) && isMa
sternodeSubset(masterNodes, validatorsAddress)
if !fullVerify || x.HookPenalty == nil || snapshotMissing || isSupe
rsetCase {
if isSupersetCase && fullVerify {
log.Warn("[verifyHeader] fullVerify: synthetic-epoch superset toler
ated (trusting header.Validators)",
"block", header.Number.Uint64(),
"localMnCount", len(masterNodes),
"headerValCount", len(validatorsAddress))
}
masterNodes = validatorsAddress
localPenalties = contracts.ExtractAddressFromBytes(header.Penalties
)
```

The subset check requires every local address to appear in the header list. An empty local list returns false:

```
func isMasternodeSubset(subset []common.Address, superset []common.
Address) bool {
if len(subset) == 0 {
return false
}
supersetMap := make(map[common.Address]struct{}, len(superset))
for _, addr := range superset {
supersetMap[addr] = struct{}{}
}
for _, addr := range subset {
if _, ok := supersetMap[addr]; !ok {
return false
```

```
}
}
return true
}
```

The header list must contain every locally computed masternode and at least one extra address, and the quorum certificate in the header must already verify. `CompareSignersLists` reports the lists as unequal. The superset branch still assigns the header field while `fullVerify` is true. Leader selection and later epoch membership use that field.

An epoch-switch block can import with a `Validators` list that `calcMasternodes` did not produce. Forming the certificate that precedes this check still requires a supermajority of vote signatures.

Assets:

- XDPoS consensus [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Severity: Medium

Recommendations

Remediation:

- In `verifyHeader`, return `ErrValidatorsNotLegit` when `CompareSignersLists` fails and `fullVerify` is true, including when `isMasternodeSubset` is true.
- Keep the header-field fallback for the case where `fullVerify` is false.
- Compare `header.Penalties` with the penalties from `calcMasternodes` after that fallback, and return `ErrPenaltiesNotLegit` on a mismatch when `fullVerify` is true.
- Keep `verifyQC` as the certificate check. It does not replace the `Validators` comparison.

F-2026-19468 - Epoch Switch Full Verification Trusts Header Fields When The V2 Snapshot Is Missing - Medium

Description:

Epoch-switch membership is taken from the gap-block V2 snapshot. `getSnapshot` returns that snapshot from cache, from the database, or from a contract-state rebuild. When all three miss, a caller with `parents == nil` can still build a snapshot from an epoch-switch header. `VerifyHeader` passes a nil parents slice, so that seed can run.

Block import calls `VerifyHeaders`. That path passes `headers[:firstV2+i]`, which is non-nil even when the slice is empty. For a regular block, `getSnapshot` treats a non-nil parents slice as bulk verification and returns `ErrV2SnapshotMissing` without seeding.

`verifyHeader` does not return that error when `fullVerify` and `snapshotMissing` is true. It logs a warning and continues. The fallback sets `masterNodes` from `GetMasternodesWithParents`. If that list is empty, it copies `header.Validators`. It then copies `header.Penalties` onto the local penalty list, so the penalty comparison matches the header field to itself. If `CompareSignersLists` still fails, `snapshotMissing` assigns `header.Validators` as well.

A node that already has the gap snapshot in cache or in the database returns that snapshot and does not take this branch. The quorum certificate in the header is verified before `calcMasternodes`.

`VerifyHeaders` passes a parents slice for every V2 header:

```
for i, header := range v2Headers {
err := c.verifyHeaderWithCache(chain, header, headers[:firstV2+i])
```

With parents set, and with no gap snapshot stored, `getSnapshot` returns:

```
return nil, fmt.Errorf("%w at gap %d (checkpoint %d): UpdateMastern
odesFromHeader did not run during gap-block import (forSigning=%t,
isGapInput=%t, parentsNil=%t, parentsLen=%d) — bulk-sync, deferred
to verifyHeader fallback", utils.ErrV2SnapshotMissing, gapNumber, c
heckpointNumber, forSigning, isGapInput, parents == nil, len(parent
s))
```

Full verification continues after that error:

```
snapshotMissing := errors.Is(err, utils.ErrV2SnapshotMissing)
if fullVerify && !snapshotMissing {
log.Error("[verifyHeader] calcMasternodes failed during normal oper
ation", "Number", header.Number, "Hash", header.Hash(), "err", err)
return err
}
if fullVerify && snapshotMissing {
log.Warn("[verifyHeader] V2 snapshot missing under fullVerify — usi
ng header-anchored fallback (A.94 checkpoint-sync-no-state aftermat
```

```
h)", "Number", header.Number, "Hash", header.Hash(), "err", err)
}
```

The fallback then reads the header fields:

```
masterNodes = x.GetMasternodesWithParents(chain, header, parents)
if len(masterNodes) == 0 {
validatorsAddress := contracts.ExtractAddressFromBytes(header.Valid
ators)
if len(validatorsAddress) > 0 {
masterNodes = validatorsAddress
}
}
localPenalties = contracts.ExtractAddressFromBytes(header.Penalties
)
```

This path requires an epoch-switch header, a missing gap snapshot, and a `VerifyHeaders` call. A stored snapshot still returns before the error. The membership check does not reject the header when the snapshot is absent and `fullVerify` is true.

An epoch-switch block can import with `Validators` and `Penalties` taken from the header because the gap snapshot was never written. Forming the certificate that precedes this fallback still requires a supermajority of vote signatures.

Assets:

- XDPoS consensus [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Severity: Medium

Recommendations

Remediation:

- In `verifyHeader`, return error when `ErrV2SnapshotMissing` and `fullVerify` is true.
- In `getSnapshot`, seed from the epoch-switch header for a `VerifyHeaders` call when the gap snapshot is absent. A non-nil parents slice is not a reason to skip that seed once `fullVerify` is true.
- Leave `header.Validators` and `header.Penalties` unread on that path.
- Keep `verifyQC` as the certificate check. It does not replace the membership comparison.

F-2026-19608 - Header Verification Accepts A Future Round With No Timeout Certificate - Medium

Description:

A V2 header carries a quorum certificate and a round. `verifyHeader` accepts the round when it is strictly greater than the certificate's round. It does not require the round to be the certificate round plus one, and it does not read a timeout certificate for the rounds in between.

The proposer has to be `masterNodes[round % Epoch % len(masterNodes)]` for the round the header claims when `fullVerify` is true. The timestamp still has to be at least the parent's time plus that round's mine period. The embedded certificate still has to match the parent and verify. `commitBlocks` still returns without committing when the parent round is not the proposed round minus one, and when the grandparent round is not the proposed round minus two. `verifyVotingRule` still returns false when the block round is not `currentRound`, so a node whose round has not moved does not vote for the header.

`InsertChain` writes the block and sets it as the head when the parent is the current head. A same-height block whose parent is the head's parent is also written, and `writeBlockAndSetHead` reorgs to the block just inserted. `NewBlock` delivers both of those shapes to `HandleProposedBlock`. A header that skips ahead is canonical until a later insert replaces it. An honest block for the round the node is actually in has the same parent and the same height, so `NewBlock` inserts it as a sibling and that insert can replace the head. `NewBlock` does not insert a block that is neither an extension of the head nor a same-height sibling.

The round check is only the lower bound:

```
if round <= quorumCert.ProposedBlockInfo.Round {
return utils.ErrRoundInvalid
}
```

The commit rule still requires consecutive rounds, and does not treat this header as finalized:

```
if *proposedBlockRound-1 != round {
return false, nil
}
```

The proposer still has to be the leader for the claimed round, and the parent certificate still has to verify. When those hold, the header is imported and can become the canonical head without a timeout certificate and without a vote in that round.

Assets:

- XDPoS consensus [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 3/5

Likelihood Rate: 3/5

Severity: Medium

Recommendations

Remediation:

- In `verifyHeader`, accept a round above the parent certificate round plus one only when a timeout certificate covers each skipped round and verifies against that epoch's active set.
- Keep the consecutive-round check in `commitBlocks`.
- Do not set a header as the canonical head when its round is not the current round and it carries no such timeout certificate.

F-2026-19609 - The Vote Lock Is The Parent Certificate, So A Certified Block Is Not Locked - Medium

Description:

`processQC` updates `highestQuorumCert` from the certificate it was given. That certificate names the block just certified. The lock is not that certificate. For a header past the V2 switch, `processQC` reads the quorum certificate stored inside the certified header and, when that inner certificate's round is higher, stores it in `lockQuorumCert`. The inner certificate names the parent of the certified block. After a certificate for block B, the lock round is the round of B's parent.

`verifyVotingRule` votes when the proposal's certificate round is greater than the lock round. Otherwise it votes when the proposal extends the locked block. A proposal whose parent is B's parent, at `currentRound`, carries the same parent certificate the lock already holds. That round is not greater than the lock. The proposal does extend the locked block, because the locked block is B's parent. `sendVote` then broadcasts the vote. `ProposedBlockHandler` runs this path for a block `InsertChain` has accepted, including a same-height sibling.

`highestVotedRound` still allows one vote in the round, so one node does not sign two proposals at the same round. `commitBlocks` still commits the grandparent only when the proposed, parent, and grandparent rounds are consecutive, and it refuses a commit at a lower round or a lower number. A vote for the sibling does not by itself finalize a second block. It does collect toward a certificate for a block that does not extend B. B can then be abandoned before the three-chain commit includes it.

The lock is assigned from the certified header's own certificate:

```
proposedBlockQuorumCert, round, _, err := x.getExtraFields(chain, p
roposedBlockHeader)
if x.lockQuorumCert == nil || (proposedBlockQuorumCert != nil && pr
oposedBlockQuorumCert.ProposedBlockInfo.Round > x.lockQuorumCert.Pr
oposedBlockInfo.Round) {
x.lockQuorumCert = proposedBlockQuorumCert
}
```

The vote rule treats an equal certificate round as an extension check:

```
if quorumCert.ProposedBlockInfo.Round > x.lockQuorumCert.ProposedBl
ockInfo.Round {
return true, nil
}
isExtended, err := x.isExtendingFromAncestor(chain, blockInfo, x.lo
ckQuorumCert.ProposedBlockInfo)
```

The proposal still has to match `currentRound`, and its proposer still has to pass header verification for that round. When it does, honest nodes vote for a block that does not extend the block named by `highestQuorumCert`.

Assets:

- XDPoS consensus [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 3/5

Likelihood Rate: 3/5

Severity: Medium

Recommendations

Remediation:

- In `processQC`, set `lockQuorumCert` to the incoming certificate, the one whose proposed block is the block just certified.
- Keep the extension check in `verifyVotingRule` against that certificate's block.
- Leave `commitBlocks` on the consecutive-round rule.

F-2026-19622 - A Vote That Completes A Certificate Latches The Next Round - Medium

Description:

`highestVotedRound` is the round this masternode has already voted. `verifyVotingRule` refuses another vote while `currentRound` is less than or equal to that value. `sendVote` sets the latch only after `voteHandler` returns nil, so a vote that the local handler drops is not recorded.

`voteHandler` adds the vote to the pool. When the pool reaches the certificate threshold and the proposed header is already local, it builds a quorum certificate and calls `processQC`. `processQC` moves `currentRound` to the certificate round plus one before `voteHandler` returns. `sendVote` then stores that new `currentRound` in `highestVotedRound`. The round that was voted is the previous one. The next round is recorded as already voted, and `verifyVotingRule` returns false for it.

A vote that does not complete a certificate leaves `currentRound` unchanged, and the latch stores the round that was voted. `VoteHandler` for a vote from another masternode calls `processQC` and does not write `highestVotedRound`. Masternodes that voted before the pool filled keep the round they voted and still vote in the next round.

The node that hits this path is the one whose own vote is the call that finds the header present and the pool at threshold. That is the ordinary last vote of a round, and any node that already held enough votes while the block was missing and then votes once the block is imported. One skipped vote does not by itself keep the next certificate from forming. If that round still certifies, `processQC` on the next certificate moves `currentRound` one past the latched value and this node votes again. If the round times out, `processTC` moves `currentRound` to the timeout round plus one, which is also past the latch.

The latch reads `currentRound` after the handler returns:

```
if err := x.voteHandler(chain, voteMsg); err != nil {
log.Error("[sendVote] Local handler error", "error", err)
return err
}
x.highestVotedRound = x.currentRound
```

The handler's certificate advances the round before that assignment:

```
if incomingQuorumCert.ProposedBlockInfo.Round >= x.currentRound {
x.setNewRound(chain, incomingQuorumCert.ProposedBlockInfo.Round+1)
}
```

The next proposal is refused while the latch matches the new round:

```
if x.currentRound <= x.highestVotedRound {
log.Info("[verifyVotingRule] Already voted this round",
"currentRound", x.currentRound,
"highestVotedRound", x.highestVotedRound)
return false, nil
}
```

The vote still has to be accepted and broadcast, and the header still has to be present when the pool crosses the threshold. When both hold, that masternode does not vote in the following round. The lock and the committed chain are unchanged.

Assets:

- XDPoS consensus [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 2/5

Likelihood Rate: 3/5

Severity: Medium

Recommendations

Remediation:

- In `sendVote`, after `voteHandler` returns nil, set `highestVotedRound` to `blockInfo.Round`.
- Do not read `currentRound` for that assignment. `processQC` may already have moved it.
- Keep the assignment after the handler returns, so a dropped vote still does not latch.

F-2026-19637 - Apothem Penalty Upgrade Releases A Penalized Signer After One Epoch Instead Of Five - Medium

Description:

Apothem activates `TIPUpgradePenalty` at block 83,600,000. The round config from round 27,360,000 sets `LimitPenaltyEpoch` to 5 and `MinimumSigningTx` to 30, and raises `MinimumMinerBlockPerEpoch` to 5. The chain config records that the canonical tip is already past that fork. Mainnet leaves `TIPUpgradePenaltyBlock` unset, so this path does not run there.

`HookPenalty` uses the upgraded mining floor. A masternode with fewer than five blocks in the epoch is added to the penalty list for the epoch switch being built. The comeback walk that follows is inside `if !chainConfig.IsTIPUpgradePenalty`. After the fork that walk does not run, and nothing replaces it. `currentConfig.LimitPenaltyEpoch` and `currentConfig.MinimumSigningTx` are never read.

`calcMasternodes` removes that one-epoch list from the candidate pool and caps the result. The next epoch switch loads the previous epoch's seated set as `preMasternodes`. A signer who was removed is not in that set, so the "no blocks mined" loop does not penalize them again. They return to the candidate pool and can be seated on the following switch.

`Prepare` writes the resulting `Validators` and `Penalties` into the epoch-switch header. Steady-state `verifyHeader` rebuilds the same lists and `CompareSignersLists` requires an equal set. Apothem's canonical epoch-switch headers are produced by XDPoSChain v2.8, whose post-upgrade branch of `HookPenalty` keeps a penalized signer in Penalties until that signer has appeared in each of the last LimitPenaltyEpoch penalty lists and has at least MinimumSigningTx signing transactions in the closing epoch. A canonical header that still excludes a signer on a later epoch switch. fails the comparison in this client with `ErrValidatorsNotLegit` or `ErrPenaltiesNotLegit`. Nodes running this client agree with each other: each writes and accepts a `Penalties` list without that signer and a `Validators` set that seats the signer again. The leader index is `round % Epoch % len(set)` over that `Validators` set.

The upgraded floor and the walk that the fork disables:

```
if chainConfig.IsTIPUpgradePenalty(number) {
minimumBlocks = currentConfig.MinimumMinerBlockPerEpoch
}
```

```
if !chainConfig.IsTIPUpgradePenalty(number) {
// comeback walk; no other branch
}
```

The condition that must hold is an Apothem epoch switch after block 83,600,000 in which a seated signer mined fewer than five blocks. When it holds, this client seats that signer again on the next switch, and steady-state verification rejects a canonical header that still keeps them out.

Assets:

- eth/hooks [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 3/5

Likelihood Rate: 3/5

Severity: Medium

Recommendations

Remediation:

- In `HookPenalty`, when `IsTIPUpgradePenalty` is true, follow the post-upgrade branch from XDPoSChain v2.8 (the `else` block after the penalty upgrade).
- Take the penalty window from `resolvePostUpgradeLimitPenaltyEpoch`: `currentConfig.LimitPenaltyEpoch` when it is greater than 0, otherwise 1. Enter the branch only when the block number is greater than `limitPenaltyEpoch * Epoch + V2.SwitchBlock`.
- Count each address across `GetPreviousPenaltyByHash` for offsets 0 through `limitPenaltyEpoch - 1`. Append an address from the offset-0 list to the penalties already collected for this epoch, unless that count equals `limitPenaltyEpoch` and the address produced at least `currentConfig.MinimumSigningTx` signing transactions in the epoch being closed.
- Scan up to `Epoch` block hashes from the epoch-switch block of the closing epoch to the parent, oldest first. Count a signing transaction only when the block hash in its payload belongs to a block in that scan whose number is divisible by `MergeSignRange` and which the scan has already reached. Count every such transaction, and recover its sender with the block's signer.
- Add a regression test that replays an Apothem epoch switch after block 83,600,000 where a penalized signer is still inside that window, and checks that `Validators` and `Penalties` match the canonical header. On devnet, run that replay on the round-50,000 config, where `LimitPenaltyEpoch` is 5.

F-2026-19638 - Block Signer And Finality RPCs Read The Seal As Addresses - Medium

Description:

`eth_getBlockSignersByNumber`, `eth_getBlockSignersByHash`, `xdc_getBlockSignersByNumber`, and `xdc_getBlockSignersByHash` take `header.Validator` and split it into 20-byte addresses. On a V2 block that field is the 65-byte proposer seal. On a co-signed V1 block it is the 65-byte co-signature of the assigned M2 validator. Sixty-five bytes yield three addresses, and the remaining five bytes are dropped. The branch that returns `header.Coinbase` runs only when `Validator` is empty. The engine recovers one signer from that signature, and on V2 that signer is the coinbase.

`findFinalityOfBlock` divides that signer count by the masternode count. Three addresses over 108 masternodes yield a finality of 2 percent. The engine records finality with `SetFinalized` when a quorum certificate commits a block. This RPC does not read `CurrentFinalBlock` or the certificate in the header extra. A committed block is still returned as a few percent.

The split of the seal:

```
for i := 0; i+common.AddressLength <= len(header.Validator); i += c
ommon.AddressLength {
var addr common.Address
copy(addr[:], header.Validator[i:i+common.AddressLength])
signers = append(signers, addr)
}
```

The percentage uses that length:

```
finality := uint((signerCount * 100) / masternodeCount)
```

These methods are registered on a default node. Every sealed block with a 65-byte `Validator` takes this path.

Assets:

- RPC [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 3/5

Likelihood Rate: 4/5

Severity: Medium

Recommendations

Remediation:

- Recover the signer from the seal the way `Author` does, and return that one address.
- For finality, report whether the block number is at or below `CurrentFinalBlock`, or whether the header's quorum certificate committed it.

F-2026-19639 - Epoch RPCs Use The Block Number Instead Of The V2 Round - Medium

Description:

After the V2 switch, the engine's epoch is `SwitchEpoch + round/Epoch`, and the epoch-switch block is the block that carried that round. Timeout rounds move that block off multiples of 900. Mainnet switches at block 80,370,000 with `SwitchEpoch` 89300.

The RPC handlers do not call the V2 engine's `GetCurrentEpochSwitchBlock`, which computes that value. `GetCurrentEpochSwitchBlock` on the wrapper divides the block number by `Epoch` and multiplies back. `GetCheckpointFromEpoch` for an epoch number of 2 or more returns `Epoch * (epoch - 1)`. Epoch 2 is therefore block 900. The V1 switch block for epoch 2 is block 1800, and the V2 switch block is not on that line once rounds and block numbers diverge.

`XDPoS_getV2BlockByNumber`, `ByHash`, and `ByHeader` fill the hash, the number, and the masternode list. `Round` stays 0 and the certificate stays empty. `GetRoundNumber` reads `ExtraFields_v2.Round` from the same header. `XDPoS_getBlockInfoByEpochNum` sets `EpochRound` to `epochNumber * Epoch` and leaves the block hash empty. For mainnet epoch 89301 that product is round 80,370,900. The engine's round for that epoch is `(epoch - SwitchEpoch) * Epoch`, which is 900. `XDPoS_getEpochNumbersBetween` emits `blockNumber / Epoch`. `XDPoS_getMissedRoundsInEpochByBlockNum` sets the epoch block to `number - number%Epoch`.

The wrapper used for the latest checkpoint:

```
epochNum := num / c.config.Epoch
switchBlock := epochNum * c.config.Epoch
return switchBlock, epochNum, nil
```

The engine method that the RPC does not call returns the round-based switch block and `SwitchEpoch + round/Epoch`.

These methods are registered on a default node, and every V2 header takes the block-number formula. Once a timeout round moves the epoch-switch block off a multiple of 900, these methods return the wrong checkpoint block, epoch number, round, and missed-round window, so explorers, dashboards, and staking tools that read the masternode list or rewards at that checkpoint read them from the wrong block. Consensus does not use these methods and is not affected.

Assets:

- RPC [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 3/5

Likelihood Rate: 3/5

Severity: Medium

Recommendations

Remediation:

- Serve epoch, checkpoint, round, and missed-round queries from `engine_v2.GetCurrentEpochSwitchBlock`, `GetRoundNumber`, and `GetBlockByEpochNumber`.
- Fill `Round` and the quorum certificate from the header extra in the V2 block RPC.

F-2026-19643 - Block Import Does Not Apply The Address Blacklist - Medium

Description:

Once the current head is at or past `BlackListHFNumber`, `LegacyPool.validateTx` rejects a transaction whose sender or recipient is in the blacklist. `CopyXDCConstants` sets that number to 38,383,838 on mainnet and 23,779,191 on Apothem. Both networks are past it. A nil recipient is not listed. The check reads the pool's current head, not the block being imported.

`StateProcessor.Process` walks the block body and does not call `IsInBlacklist`. A transaction to `0x89` after `TIPSigning` goes through `ApplySigningTransaction`, which advances the sender nonce and does not run the EVM. A transaction to `0x92` or `0x93` inside the empty-transaction window goes through `ApplyEmptyTransaction`. Every other transaction is converted to a message and applied, so the nonce advances and a value transfer or a gas payment changes balances. The miner's `applyTransaction` uses the same three paths and does not call `IsInBlacklist` either. The miner fills the block from the pool, so a block built that way does not contain these transactions. A body that already contains one is still imported.

The pool check that import does not repeat:

```
if common.IsInBlacklist(&from) {
return fmt.Errorf("%w: sender %v", ErrBlacklisted, from.Hex())
}
if common.IsInBlacklist(tx.To()) {
return fmt.Errorf("%w: recipient %v", ErrBlacklisted, tx.To().Hex()
)
}
```

The import loop applies every transaction in the body:

```
for i, tx := range block.Transactions() {
// signing-tx and empty-tx shortcuts, then
// TransactionToMessage and ApplyTransactionWithEVM
}
```

The condition that must hold is a block at or past `BlackListHFNumber` whose body contains a listed sender or recipient. A block producer can include such a transaction directly, because the blacklist is checked only when a transaction enters the pool. Block import has no equivalent check, so an honest node running this client cannot identify the transaction as one that must not be executed. The node treats the block as valid, applies the transfer, and a masternode running this client votes for the block.

Assets:

- Core [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 4/5

Likelihood Rate: 2/5

Severity: Medium

Recommendations

Remediation:

- In `Process`, reject the block when its number is at or past `BlackListHFNumber` and `IsInBlacklist` is true for the sender or the recipient. Apply that check before the signing-transaction path, the empty-transaction path, and `ApplyTransactionWithEVM`.
- Apply the same check in the miner before a transaction is sealed.

F-2026-19723 - Bulk Sync Canonizes A Block Whose State Root Does Not Match Execution - Medium

Description:

`ValidateState` compares the state root produced by executing the block with `header.Root`. It also compares gas used, the receipt root, and the bloom. On a failure `ProcessBlock` normally reports the block and returns. While `canMaskMissingState` is true, that return does not happen. `ProcessBlock` logs, sets `checkpointSyncNoState`, replaces the executed state with the empty root, and still calls `writeBlockAndSetHead`. The block becomes the canonical head. The stored header keeps the root it was imported with. The trie committed for the block is the empty root.

`canMaskMissingState` is true when `XdcBulkSyncMode` is set, the chain has an XDPoS config, the node is not archive, and `NoStateMaskingForbidden` is unset. A default node sets `XdcBulkSyncMode` when `xdcSyncer` starts. `NoStateMaskingForbidden` is set from the mining flag. A node that is not mining does not set it.

Once the flag is set, a later block skips `Process` and `ValidateState`. That block is also written as the head. `VerifyHeaders` still runs before `ProcessBlock`. It does not execute the body. `ClearCheckpointSyncNoState` leaves the flag set when the head's announced root is not in the trie. The comment on that refusal says the database was imported without execution and must be resynced.

Followers in bulk sync share this branch, so they accept the same block. A node that is not in bulk sync, an archive node, and a mining node still return the `ValidateState` error.

The mask replaces a failed state check and continues:

```
if bc.canMaskMissingState() {
log.Warn("BlockChain: A.94 auto-recover on ValidateState mismatch d
uring bulk sync",
"block", block.NumberU64(), "err", err)
bc.storeCheckpointSyncNoState(true)
rawdb.WriteCheckpointSyncNoState(bc.db, true)
res = &ProcessResult{Receipts: types.Receipts{}}
statedb, err = state.New(types.EmptyRootHash, sdb)
```

The head write is the same call used for a block that passed:

```
status, err = bc.writeBlockAndSetHead(block, res.Receipts, res.Logs
, statedb, false)
```

The condition that must hold is a non-archive, non-mining follower with `XdcBulkSyncMode` set, and a block that passes header verification while `ValidateState` returns an error. When it holds, that block is the canonical head, and the following blocks are imported without execution.

Assets:

- Mining / sync [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 3/5

Likelihood Rate: 3/5

Severity: Medium

Recommendations

Remediation:

- In `ProcessBlock`, return the `ValidateState` error. Do not replace the state and do not call `writeBlockAndSetHead` for that block.
- Leave the missing-parent branch, where `state.New` fails before execution, as the only path that may set `checkpointSyncNoState`. A root that execution already rejected is not that case.

F-2026-19726 - Round Timeout Uses The Round Zero Period - Medium

Description:

The countdown that asks a masternode for a timeout is created from the round-0 row. `CurrentConfig` is that row. `createEngine` copies `TimeoutPeriod` and the exponential base from it into `NewExpCountDown`, and the period is stored as `baseDuration`. The configured maximum exponent is 0. The copy accepts that field only when it is greater than 0, so the countdown keeps its default maximum of 6. The base is 1, and 1 raised to any exponent is 1, so the wait is the stored period.

`UpdateConfig` replaces `CurrentConfig` from `AllConfigs` for a later round. `UpdateParams` is its only caller, and `UpdateParams` is the only function that then calls `SetParams` on the countdown. The running node never calls `UpdateParams`. Engine tests call it. Import, mining, `initial`, and `setNewRound` do not.

`setNewRound` stores the new round and calls `Reset`. `Reset` waits `baseDuration` times the exponential factor and does not read `Config(round)`. `initial` calls `Reset` on the same countdown and does not call `SetParams`.

`OnCountdownTimeout` sends the timeout when `allowedToSend` accepts the current header. `verifyTC` checks the certificate signatures, the signer set, and the quorum threshold. It does not compare the wait with `Config(round).TimeoutPeriod`, so a certificate produced on the round-0 period is accepted. The same handler reads `Config(currentRound)` for the sync-info threshold. On the active mainnet and Apothem rows that threshold is 3, the same value as round 0.

Mainnet round 0 sets the period to 30 seconds. The row at round 17,579,700 sets it to 60 seconds. The V2 switch block is round 0, at block 80,370,000, and each later block carries a round greater than its parent certificate. The mainnet head recorded on 11 May 2026 is about 102,445,681. That height is past block 97,949,700, so the round is past 17,579,700. The active period is 60 seconds. The countdown waits 30 seconds.

Apothem round 0 sets the period to 60 seconds. The row at round 15,000,000 sets it to 10 seconds, and the row at round 27,360,000 keeps 10 seconds. The Apothem configuration records a canonical round of 27,460,297, which selects the 10-second row. The countdown waits 60 seconds.

Devnet's round-0 period and the row at round 50,000 are both 10 seconds. The devnet wait matches its configured period.

Masternodes running this client share that countdown and emit the timeout together when the round is still open at the end of the round-0 period. A round that receives a quorum certificate first is advanced by `setNewRound`, and the node does not send the timeout.

`createEngine` takes the period from the round-0 row:

```
if config.V2 != nil && config.V2.CurrentConfig != nil {
timeoutPeriod = config.V2.CurrentConfig.TimeoutPeriod
if config.V2.CurrentConfig.ExpTimeoutConfig.Base > 0 {
expBase = config.V2.CurrentConfig.ExpTimeoutConfig.Base
}
if config.V2.CurrentConfig.ExpTimeoutConfig.MaxExponent > 0 {
maxExponent = int(config.V2.CurrentConfig.ExpTimeoutConfig.MaxExpon
ent)
}
}
duration := time.Duration(timeoutPeriod) * time.Second
timeoutTimer, err := countdown.NewExpCountDown(duration, expBase, m
axExponent)
```

`setNewRound` restarts that same duration:

```
x.currentRound = round
x.timeoutCount = 0
x.timeoutWorker.Reset(chain, uint64(x.currentRound), uint64(x.highe
stQuorumCert.ProposedBlockInfo.Round))
```

On mainnet the timeout is sent after 30 seconds while the active period is 60 seconds. On Apothem the timeout is sent after 60 seconds while the active period is 10 seconds.

Assets:

- XDPoS consensus [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 3/5

Likelihood Rate: 3/5

Severity: Medium

Recommendations

Remediation:

- In `setNewRound`, select `Config(round)`, call `UpdateConfig` for that round, and call `SetParams` before `Reset`.
- In `initial`, apply the head round the same way before `Reset`. `createEngine` has no header, so it cannot select that row.

F-2026-19773 - Chain Reorganisation Can Revert A Committed Block - Medium

Description:

XDPoS 2.0 commits a block when three linked blocks carry consecutive rounds and the newest one is certified. `commitBlocks` records that block in `highestCommitBlock` and calls `HookCommitBlock`, which the node wires to `SetFinalized`. A committed block is final by the protocol's definition, so no honest node may later drop it from the canonical chain.

In the reference client XDPoSChain, `reorg` reads `GetLatestCommittedBlockInfo` after it finds the common ancestor. When the common ancestor is below the committed number, it takes the old-chain block at that number and returns `stop reorg, blockchain is under forking attack` when that block's hash equals the committed hash. A different hash at that number is only logged, and the reorg proceeds. The guard inspects only the old chain and never checks whether the new chain carries the committed block.

The audited client carries no such guard. `writeBlockAndSetHead` and `writeKnownBlock` call `reorg` whenever the inserted block's parent is not the current head, and then write the inserted block as the head. `reorg` swaps the chains without reading the committed block, and nothing in the audited client calls `GetLatestCommittedBlockInfo`. `SetFinalized` stores whatever header it receives. `commitBlocks` requires only a higher round and number than `highestCommitBlock`, not descent from it.

The head update in `writeBlockAndSetHead`:

```
currentBlock := bc.CurrentBlock()

// Reorganise the chain if the parent is not the head block
if block.ParentHash() != currentBlock.Hash() {
if err := bc.reorg(currentBlock, block.Header()); err != nil {
return NonStatTy, err
}
}

// Set new head.
bc.writeHeadBlock(block)
```

The reference client guard in `reorg`:

```
// Ensure XDPoS engine committed block will be not reverted
if xdpos, ok := bc.Engine().(*XDPoS.XDPoS); ok {
latestCommittedBlock := xdpos.EngineV2.GetLatestCommittedBlockInfo(
)
if latestCommittedBlock != nil {
cmp := commonBlock.Number.Cmp(latestCommittedBlock.Number)
if cmp < 0 {
for _, oldBlock := range oldChain {
if oldBlock.Number.Cmp(latestCommittedBlock.Number) == 0 {
if oldBlock.Hash() != latestCommittedBlock.Hash {
log.Error("Impossible reorg, please file an issue", "OldNum", oldBl
ock.Number, "OldHash", oldBlock.Hash().Hex(), "LatestCommittedHash"
, latestCommittedBlock.Hash.Hex())
} else {
```

```
log.Warn("Stop reorg, blockchain is under forking attack", "OldComm
ittedNum", oldBlock.Number, "OldCommittedHash", oldBlock.Hash().Hex
())
return fmt.Errorf("stop reorg, blockchain is under forking attack.
OldCommitted num %d, hash %s", oldBlock.Number, oldBlock.Hash().Hex
())
}
}
}
// Other Code
```

Gossip alone cannot reach a committed block, because the `NewBlock` handler accepts only a direct child of the head or a same-height sibling. A batch whose first block has a stored parent below the committed block does reach `reorg`, but every header in it must still pass the consensus verification in force for its height, including the ancestry, proposer, signature, certificate, state root and execution checks. The missing guard is therefore exploitable only when a competing branch that is otherwise accepted is inserted below the committed block. Full synchronization inserts a non-extending batch without an anchor check (F-2026-19636).

When such a branch is inserted, the node drops the committed block and the blocks after it from its canonical chain. They stay in storage, but lookups by block number and by transaction hash no longer return them. The `finalized` tag of `eth_getBlockByNumber` keeps returning the dropped block, because nothing clears the finalized marker. Wallets, bridges and exchanges that treat finalized blocks as irreversible see finalized transactions disappear.

Assets:

- XDPoS consensus [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 4/5

Likelihood Rate: 2/5

Severity: Medium

Recommendations

Remediation:

- In `reorg`, read `GetLatestCommittedBlockInfo` through the XDPoS engine. Return an error when the common ancestor is below the committed block number and the old-chain block at that number carries the committed hash, as the reference client does.
- Reject as well a reorg whose new chain reaches below the committed number and does not carry the committed hash at that number. The reference client does not inspect the new chain.

F-2026-19459 - Empty Transaction Dispatch Imports Unsigned TradingState Payloads Without Nonce - Low

Description:

Ordinary block execution converts each transaction to a message, which recovers the sender, before the EVM runs. While `IsTIPXDCXReceiver` is true, a transaction whose destination is `TradingStateAddrBinary` (`0x92`) or `XDCXLendingAddressBinary` (`0x93`) is handled by `ApplyEmptyTransaction`. The loop then continues. `TransactionToMessage` does not run, so sender recovery, the nonce check, and the nonce increment do not run.

`ApplyEmptyTransaction` finalises the state, appends one log addressed to the destination, and returns a successful receipt with zero gas used. The log is part of the receipt. It is not an account-state write. The sender nonce is left unchanged.

`IsSpecialTransaction` is true only for `BlockSignersBinary` and `RandomizeSMCBinary`. A transaction to `0x92` or `0x93` is not in that set, so the pool still requires a signature. Block import does not.

`IsTIPXDCXReceiver` is true from `TIPXDCX` until `TIPXDCXReceiverDisable`. The disable block itself is not included, because `isBlockForked` is true when the head is at or after the fork. On mainnet the window is block 38,383,838 through block 80,370,899. On Apothem it is block 23,779,191 through block 66,824,999. Both networks are past the disable block, so a new block on those chains does not take this path. Re-execution of a block inside the window still does. Chain IDs 551, 5151, 5050, and 5551 set `TIPXDCXReceiverDisable` to `1e17`, so the dispatch stays active for every reachable block.

The same function is used when the miner applies a transaction to one of those destinations. An in-turn producer can place an unsigned payload, or a payload already included, into the block body. Import accepts it and the receipts root includes the zero-gas receipt. The sender nonce does not move, so the same payload can be included again.

While `IsTIPXDCXReceiver` is true, destination matching skips message conversion:

```
if config.IsTIPXDCXReceiver(blockNumber) {
if to := tx.To(); to != nil && (*to == common.TradingStateAddrBinar
y || *to == common.XDCXLendingAddressBinary) {
receipt, err := ApplyEmptyTransaction(config, statedb, blockNumber,
blockHash, tx, gp.Used())
if err != nil {
return nil, fmt.Errorf("could not apply empty tx %d [%v]: %w", i, t
x.Hash().Hex(), err)
}
receipts = append(receipts, receipt)
allLogs = append(allLogs, receipt.Logs...)
continue
```

```
}
}
```

The empty receipt carries one log and zero gas:

```
emptyLog := &types.Log{
BlockNumber: blockNumber.Uint64(),
}
if to := tx.To(); to != nil {
emptyLog.Address = *to
}
statedb.AddLog(emptyLog)

receipt := &types.Receipt{
Type: tx.Type(),
PostState: root,
Status: types.ReceiptStatusSuccessful,
CumulativeGasUsed: cumulativeGasUsed,
TxHash: tx.Hash(),
GasUsed: 0,
}
```

The receiver check is the trading fork and not yet the disable fork:

```
return isBlockForked(common.TIPXDCX, num) && !isBlockForked(common.
TIPXDCXReceiverDisable, num)
```

Assets:

- Core [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 3/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

- In `Process`, call `TransactionToMessage` for a transaction to `TradingStateAddrBinary` or `XDCXLendingAddressBinary` before `ApplyEmptyTransaction`.
- Reject the transaction when sender recovery fails.
- Increment that sender's nonce inside `ApplyEmptyTransaction`.
- On mainnet and Apothem, keep the zero-gas receipt for blocks from `TIPXDCX` through `TIPXDCXReceiverDisable`.
- On chain IDs 551, 5151, 5050, and 5551, apply the sender and nonce checks at every block. `TIPXDCXReceiverDisable` is `1e17` on those chains.

F-2026-19462 - V1 Checkpoint Validator Verification Never Rejects Populated Validators Field - Low

Description:

Full V1 header verification calls `HookVerifyMNs` from `verifyCascadingFields` on an epoch-checkpoint header. The hook asks `HookValidator` for the expected `Validators` bytes. Block 3,464,000 is the `TIPRandomize` fork on mainnet and Apothem, the block at which masternode order is taken from the randomize contract. `HookValidator` does not call `IsTIPRandomize`. It compares the header number with the literal `3464000`. From that block onward it does not read the randomize contract. A non-empty `Validators` field is returned unchanged.

`HookVerifyMNs` compares that return value with `header.Validators`. Both sides are the same field, so every non-empty field matches. `HookValidator` returns a nil error on this path, and `HookVerifyMNs` then returns nil.

The hook runs for a V1 header, a block at or before the V2 switch, and only when the block number is a multiple of the epoch length. Block 3,464,000 itself is not an epoch checkpoint. The first checkpoint in this branch is block 3,464,100. Mainnet switches at block 80,370,000. Apothem switches at block 56,828,700.

Bulk sync does not take this path. It verifies V1 headers in `verifyHeaderLightV1`, which does not call `HookVerifyMNs`. A header past the switch block is verified on the V2 path, which does not call the hook. On the full V1 path, `verifySeal` still checks the proposing signature.

An epoch checkpoint in that window can carry any non-empty `Validators` bytes. The hook accepts them.

When the field is empty from block 3,464,000 onward, `HookValidator` writes the signer addresses in list order into the field and returns those bytes. The comparison then matches the bytes just written. That branch also does not read the randomize contract.

From block 3,464,000 onward, a populated field is returned as the computed value:

```
if len(header.Validators) > 0 {
return header.Validators, nil
}
```

Verification compares that result with the same field and returns nil:

```
expected, err := adaptor.HookValidator(header, signers)
if err != nil {
return err
}
if len(expected) > 0 && len(header.Validators) > 0 {
if !bytes.Equal(header.Validators, expected) {
```

```
log.Warn("[V1 HookVerifyMNs] Validators field mismatch (non-fatal i
n sync mode)",
"number", number,
"expectedAddrCount", len(expected)/common.AddressLength,
"gotAddrCount", len(header.Validators)/common.AddressLength)
}
}
return nil
```

A block after the V2 switch leaves V1 verification:

```
if c.config.V2 != nil && c.config.V2.SwitchBlock != nil && number >
c.config.V2.SwitchBlock.Uint64() {
return c.verifyHeaderV2(chain, header, parents)
}
```

On the V1 path, cascading verification calls the hook:

```
if c.HookVerifyMNs != nil {
if err := c.HookVerifyMNs(header, signers); err != nil {
return err
}
}
```

Bulk sync uses the light V1 check for headers at or before the switch:

```
if bulkSync {
err = c.verifyHeaderLightV1(chain, headers[i], headers[:i])
} else {
err = c.verifyHeaderWithCache(chain, headers[i], headers[:i])
}
```

Assets:

- eth/hooks [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

- In `HookValidator`, from block 3,464,000 (`TIPRandomize` on mainnet and Apothem) onward, compute the `Validators` bytes from the randomize contract. Do not return `header.Validators` as that result.
- In `HookVerifyMNs`, return an error when the computed bytes differ from `header.Validators`.
- Keep that comparison on the full V1 path. `verifyHeaderLightV1` does not perform it.

F-2026-19467 - Block Gas Accounting Does Not Bound BlockSigners Transactions - Low

Description:

After `TIPSigning`, a transaction to `BlockSignersBinary` (`0x89`) is applied by `ApplySigningTransaction`. On mainnet and Apothem that fork is block 3,000,000, and both networks are past it. The function recovers the sender, checks the nonce, increments the nonce, and returns a successful receipt with `GasUsed` 0. It does not subtract from the block gas pool. `Process` then continues. The miner uses the same function and does not subtract either. It sets the header `GasUsed` from the pool, so a body of only these transactions records gas used of zero.

Header verification rejects the block when `GasUsed` is greater than `GasLimit`. Zero is not greater, so that comparison does not reject the body.

The packing loop stops when the remaining pool is below `TxGas`. It also skips a transaction whose declared gas is above the remaining pool. Neither check shrinks the pool for a `BlockSigners` transaction, so a later one still fits. `Pending` keeps a special transaction when the miner filter has a non-zero minimum tip, because the tip comparison exempts `IsSpecialTransaction`. The stock producer therefore packs these transactions until `txFitsSize` fails.

`ValidateBody` applies `MaxBlockSize` only when `IsOsaka` is true. `XDCMainnetChainConfig` does not set `OsakaTime`, so that size check does not run. The miner still stops at `txFitsSize`, which uses `MaxBlockSize`. The eth protocol rejects a message above `maxMessageSize` (10 MiB). Those checks bound encoded size. They do not bound the count by `GasLimit`.

`ApplySigningTransaction` does not check that the sender is a masternode. A signature and a matching nonce are still required.

`Process` dispatches `BlockSigners` after `TIPSigning` and does not subtract gas:

```
if to := tx.To(); to != nil && *to == common.BlockSignersBinary &&
config.IsTIPSigning(blockNumber) {
receipt, err := ApplySigningTransaction(config, statedb, blockNumbe
r, blockHash, tx, gp.Used())
if err != nil {
return nil, fmt.Errorf("could not apply signing tx %d [%v]: %w", i,
tx.Hash().Hex(), err)
}
receipts = append(receipts, receipt)
allLogs = append(allLogs, receipt.Logs...)
continue
}
```

The receipt records zero gas and leaves the cumulative counter unchanged:

```
receipt := &types.Receipt{
Type: tx.Type(),
PostState: root,
Status: types.ReceiptStatusSuccessful,
CumulativeGasUsed: cumulativeGasUsed,
TxHash: tx.Hash(),
GasUsed: 0,
}
```

Header verification only rejects used gas above the limit:

```
if header.GasUsed > header.GasLimit {
return fmt.Errorf("gas used exceeded gaslimit, gas used: %d, gas li
mit: %d", header.GasUsed, header.GasLimit)
}
```

The miner stops the loop on remaining gas, and skips one transaction whose declared gas does not fit. This path does not reduce the pool:

```
if env.gasPool.Gas() < params.TxGas {
break
}

if env.gasPool.Gas() < ltx.Gas {
txs.Pop()
continue
}
```

Assets:

- Core [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

- In `ApplySigningTransaction`, subtract the transaction's gas from the block gas pool, and set the header `GasUsed` from that pool.
- Keep `GasUsed` 0 on the receipt. The pool debit is the bound header verification already checks.
- In `ValidateBody`, reject a body whose encoded size is above `MaxBlockSize` when `OsakaTime` is unset.
- Keep `txFitsSize` on the miner. The import check is the one a block from another producer has to pass.

F-2026-19469 - Quorum Certificate Forensics Is Never Invoked After processQC - Low

Description:

`processQC` updates the highest quorum certificate, the lock certificate, the three-chain commit, and the current round. It returns after that update. The forensics type is never constructed on the live engine, and none of its entry points run.

`ProcessForensics` is documented as the step that runs once `processQC` succeeds. `ForensicsMonitoring` calls it and then stores the committed certificates. `ProcessVoteEquivocation` and `DetectEquivocationInVotePool` report a vote that does not extend the committed chain, or two votes from one signer in one round. `NewForensics` is reached from tests only. `createEngine` fills round state, the vote pool, and the timeout pool, and does not allocate a `Forensics` value. No production caller invokes `ForensicsMonitoring`, `ProcessForensics`, `ProcessVoteEquivocation`, or `DetectEquivocationInVotePool`.

The vote pool key includes the proposed block hash. Two votes that share a signer and a round, and name different hashes, occupy two pool entries. `processQC` replaces `highestQuorumCert` only when the incoming round is strictly greater. The lock is the quorum certificate inside the proposed block header, and it updates when that certificate's round is strictly greater than the current lock. Commit still follows the three-chain rule for the certificate that was processed. Those paths do not publish a `ForensicsEvent`.

`SendForensicProof` and `SendVoteEquivocationProof` send that event on the forensics feed. The feed has no production subscriber, because the engine never builds the object that owns it.

A certificate still passes `verifyQC` on the import path before `processQC` runs. The missing call does not admit an extra certificate. A pair of certificates that diverge, or a vote that does not extend the committed certificates, produces no proof and no event.

The engine constructor does not create forensics:

```
engine := &XDPoS_v2{
timeoutPool: utils.NewPool(),
votePool: utils.NewPool(),
```

`processQC` finishes on the round update:

```
if incomingQuorumCert.ProposedBlockInfo.Round >= x.currentRound {
x.setNewRound(chain, incomingQuorumCert.ProposedBlockInfo.Round+1)
}
return nil
```

The forensics entry point states when it is supposed to run:

```
Triggered once processQC is successfully.
```

The vote pool separates hashes in one round:

```
return fmt.Sprint(v.ProposedBlockInfo.Round, ":", v.GapNumber, ":",
v.ProposedBlockInfo.Number, ":", v.ProposedBlockInfo.Hash.Hex())
```

Honest nodes share this engine, so none of them emit the proof. Commit still uses the certificate the engine already accepted. A diverging certificate, and two votes from one signer for two hashes, produce no forensics proof.

Assets:

- XDPoS consensus [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

- Add a `Forensics` field on the V2 engine and construct it in `createEngine`.
- After `processQC` succeeds, call `ForensicsMonitoring` with the committed headers and the incoming quorum certificate.
- After a vote is accepted into the vote pool, call `ProcessVoteEquivocation` and `DetectEquivocationInVotePool`.
- Subscribe to the forensics feed from the process that receives the proof.
- Keep `verifyQC` as the import check. It does not replace the forensics call.

F-2026-19596 - Timeout Certificates Count Penalized Signers Toward The Active Set Quorum - Low

Description:

A timeout certificate records that the active masternode set for the round has timed out. The numeric threshold uses that active count. Signature membership uses a larger set: the active masternodes plus the recorded penalties. A penalized address counts toward a quorum whose size excludes that address.

`CandidatePool` returns the recorded candidates unchanged when the penalty list is empty. When the penalty list is non-empty, it returns those candidates plus every penalized address that was not already in the list.

`VerifyTimeoutMessage` recovers the signer and accepts the timeout when that address is in `CandidatePool`. `timeoutHandler` then counts every pooled timeout against `MasternodesLen`, the active length:

```
isThresholdReached := float64(numberOfTimeouts) >= float64(epochInf
o.MasternodesLen)*certThreshold
```

`onTimeoutPoolThresholdReached` copies every pooled signature into the certificate and calls `processTC`. Signers outside the active masternode list stay in that certificate. `processTC` replaces `highestTimeoutCert` when the incoming round is strictly greater. When that round is at least the current round, it sets the current round to the certificate round plus one.

`verifyTC` uses the same two sets. The count is the active length:

```
if float64(len(signatures)) < float64(epochInfo.MasternodesLen)*cer
tThreshold {
return utils.ErrInvalidTCSignatures
}
```

Each signature is then checked against the candidate pool:

```
verified, _, err := x.verifyMsgSignature(signedTimeoutObj, sig, poo
l)
```

After a vote is pooled, `verifyVotes` removes any signer that is not in the current masternode list, and the quorum certificate is built from the signatures that remain. Timeout assembly keeps every pooled signature.

`processTC` does not change `lockQuorumCert`. `verifyVotingRule` allows a vote when the block's quorum certificate has a higher round than that lock. Otherwise the block has to extend the lock. `verifyQC` checks a block certificate against `epochInfo.Masternodes`. This timeout path does not commit a different block. It lets a timeout certificate advance the round when fewer

active masternodes have signed than `MasternodesLen` times the certificate threshold.

Honest nodes share this membership list and this threshold, so they accept the same certificate. The penalty list has to be non-empty. When it is empty, `CandidatePool` and the active set are the same list and the threshold matches the signers.

Assets:

- XDPoS consensus [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

- Check timeout signatures against the active masternode list, the same list whose length is `MasternodesLen`.
- In `onTimeoutPoolThresholdReached`, drop signers that are not in that list before counting them and before building the certificate.
- Keep `CandidatePool` for the recorded penalty list. Use the active masternode list as the quorum membership set.

F-2026-19602 - V1 Checkpoint Extra Replaces The Signer Set And Authorizes That Sealer - Low

Description:

Full V1 verification of an epoch checkpoint compares the address list in `header.Extra` with the signer set taken from the parent snapshot, after penalties. On a mismatch, an empty list returns `errInvalidCheckpointSigners`. A non-empty list does not. `verifyCascadingFields` replaces `snap.Signers` with that list, writes the snapshot, puts the same object back in the recents cache under the parent hash, and continues. The log line calls this sync mode. The branch does not read `xdcFullVerify` and does not check bulk sync.

`verifySeal` then loads the snapshot for the same parent hash. The recents cache returns the replaced set. The creator recovered from the Extra seal is authorized when that address is in the replaced set. For a checkpoint, `GetMasternodesWithParents` returns the same Extra addresses, so the fallback membership check reads the list being verified.

`calcDifficulty` still returns a positive integer when the creator is absent from the parent masternode list. `YourTurn` leaves the creator index at `-1` and returns no error, and `Hop` with that index still yields a difficulty the header can carry. `verifySeal` rejects a difficulty only when the calculated value is non-zero and differs. The Extra seal covers the signer bytes. After the replacement, `verifySeal` authorizes the recovered creator only when that address is in the Extra list.

The replaced object stays cached under the parent hash. A later snapshot walk that stops on that cache entry copies the replaced set forward. `loadSnapshot` reads a stored snapshot only at a gap block. The parent of a checkpoint is not a gap block (`Epoch` is 900 and `Gap` is 450), so a restart does not load this record back through the gap path. Blocks already imported on the replaced set stay in the chain.

Bulk sync does not take this branch. It uses `verifyHeaderLightV1`, which checks Extra length and does not compare signers. A header with number greater than the V2 switch leaves V1 verification. Mainnet switches at block 80,370,000 and Apothem at block 56,828,700. When a V1 checkpoint is the current head, `NewBlock` inserts a same-height sibling and `writeBlockAndSetHead` makes that sibling the head. `NewBlock` does not insert a checkpoint that is no longer the head.

The mismatch is accepted and the snapshot is overwritten:

```
if len(masternodesFromCheckpointHeader) > 0 {
newSigners := make(map[common.Address]struct{}, len(masternodesFrom
CheckpointHeader))
for _, mn := range masternodesFromCheckpointHeader {
newSigners[mn] = struct{}{}
```

```
}
snap.Signers = newSigners
if err := snap.store(c.db); err != nil {
log.Warn("Failed to persist corrected checkpoint snapshot", "number
", number, "err", err)
}
c.recents.Add(snap.Hash, snap)
signers = masternodesFromCheckpointHeader
}
```

A block past the switch does not enter this function:

```
if c.config.V2 != nil && c.config.V2.SwitchBlock != nil && number >
c.config.V2.SwitchBlock.Uint64() {
return c.verifyHeaderV2(chain, header, parents)
}
```

The creator still has to recover from the Extra seal, the header still has to meet the timestamp and checkpoint coinbase rules, and the block's state root still has to match execution. When those hold, the checkpoint is imported and the cached signer set is the Extra list.

Assets:

- XDPoS consensus [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 4/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

- In `verifyCascadingFields`, return `errInvalidCheckpointSigners` when the checkpoint Extra list differs from the computed signer set.
- Do not write that list into `snap.Signers`, the database, or the recents cache.
- In `verifySeal`, authorize the creator from the parent snapshot. Do not treat `GetMasternodesFromCheckpointHeader` on the header under verification as proof that the creator is in the set.

F-2026-19612 - Timeout Gap Number Selects The Signer Set Independently Of The Round Epoch - Low

Description:

A timeout names a round and a gap number, and the signature covers both. `VerifyTimeoutMessage` loads the snapshot at `timeout.GapNumber` and accepts the signer when that address is in the snapshot's `CandidatePool`. `timeoutHandler` then counts every pooled timeout for that round and gap against `MasternodesLen` from the epoch of the current head. `verifyTC` uses the same split. Membership is `CandidatePool` of the certificate's gap. The count is `MasternodesLen` from `getTCEpochInfo`, which walks to the epoch that contains the certificate round.

No check requires the gap number to be the gap of that epoch. `getSnapshot` with `forSigning` set treats the supplied number as a gap block. When contract state at that header exists, `rebuildGapSnapshotFromState` stores the full candidate list from that state and does not cap it at `MaxMasternodes`. `CandidatePool` returns that stored list. When state is missing, the forward seed starts at `gapNumber + Gap - 1` and stores the validator list from the next epoch-switch header at or after that point. That list is the active set recorded on the header. Either list can differ from the active set of the certificate's round.

`verifyVotes` drops a vote signer who is not in the current masternode list before a quorum certificate is built. `onTimeoutPoolThresholdReached` copies every pooled signature into the timeout certificate.

`processTC` sets `currentRound` to the certificate round plus one when that round is at least the current round. It does not update `lockQuorumCert`.

Honest nodes that resolve the same snapshot accept the same certificate. A node that still has state at the gap rebuilds the full candidate list from that state. A node without that state forward-seeds from the epoch-switch header at or after `gapNumber + Gap - 1`, and that list can differ. Each signature still has to recover to an address in the snapshot that node loaded. The number of signatures still has to reach that epoch's `MasternodesLen` times `CertThreshold`. On mainnet `CertThreshold` is `0.667`. When `MasternodesLen` is 108, `108 * 0.667` is 72.036, so the certificate needs 73 signatures. Those addresses do not have to belong to the active set of the epoch that contains the round. Timeouts for a different gap stay in a different pool, so signatures on the current gap do not fill this certificate.
A timeout certificate built from such signatures is accepted by `verifyTC` and passed to `processTC`, which moves `currentRound` to the certificate round plus one on every node that accepts it. The certificate can therefore be completed by standby candidates and penalized addresses that the gap

snapshot's `CandidatePool` returns, with fewer than two thirds of the masternodes that are active for that round.

The ingress check uses the gap snapshot:

```
pool := snap.CandidatePool()
verified, signer, err := x.verifyMsgSignature(
types.TimeoutSigHash(&types.TimeoutForSign{
Round: timeoutMsg.Round,
GapNumber: timeoutMsg.GapNumber,
}),
timeoutMsg.Signature,
pool,
)
```

The handler counts that pool against the current epoch:

```
epochInfo, err := x.getEpochSwitchInfo(chain, chain.CurrentHeader()
, chain.CurrentHeader().Hash())
certThreshold := x.config.V2.Config(uint64(timeout.Round)).CertThre
shold
isThresholdReached := float64(numberOfTimeouts) >= float64(epochInf
o.MasternodesLen)*certThreshold
```

Certificate verification uses the gap snapshot for membership:

```
pool := snap.CandidatePool()
```

The count is the epoch length for the certificate round:

```
if float64(len(signatures)) < float64(epochInfo.MasternodesLen)*cer
tThreshold {
return utils.ErrInvalidTCSignatures
}
```

Each signature is then checked against that pool:

```
verified, _, err := x.verifyMsgSignature(signedTimeoutObj, sig, poo
l)
```

Assets:

- XDPoS consensus [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

- Derive the gap from the epoch that contains the timeout round, and reject a timeout whose `GapNumber` differs.
- Verify timeout signatures against the active masternode list of that epoch, the same list whose length is the threshold.
- In `timeoutHandler`, drop signers outside that list before the threshold count. In `onTimeoutPoolThresholdReached`, copy only the signatures that remain.

F-2026-19614 - V1 Full Seal Logs The M2 Co-Sign And Does Not Reject It - Low

Description:

Past the first epoch, a V1 block is supposed to carry `header.Validator` as the co-signature of the M2 assigned to the creator. `verifySeal` runs that comparison only when `xdcFullVerify` is true and the block number is greater than `Epoch`. An empty `Validator`, a recovery failure, or a recovered address other than the assigned M2 is logged as `[DIAG-1c]`. The function then returns nil. `errNoValidatorSignature` and `errFailedDoubleValidation` are not returned on this path.

`verifyHeaderLightV1` returns `errNoValidatorSignature` when `Validator` is empty, the number is past the first epoch, and the chain id is mainnet (50). It does not recover `Validator` and does not compare it with the assigned M2. A non-empty wrong signature passes that light check as well.

The Extra seal and the signer-set check still run in `verifySeal`. A creator outside that set is still `errUnauthorized`. This path admits an in-set creator whose `Validator` is empty or recovers to someone other than the assigned M2.

The comparison logs and does not return the error:

```
fullVerify := xdcFullVerify()
if fullVerify && number > c.config.Epoch {
switch {
case len(header.Validator) == 0:
log.Warn("[DIAG-1c] empty header.Validator past epoch (legacy→ErrNo
ValidatorSignature)", "number", number, "creator", creator)
default:
validator, vErr := c.RecoverValidator(header)
assigned, aErr := c.GetValidator(creator, chain, header)
switch {
case vErr != nil:
log.Warn("[DIAG-1c] RecoverValidator failed", "number", number, "er
r", vErr)
case aErr != nil:
log.Warn("[DIAG-1c] GetValidator(M2) failed", "number", number, "cr
eator", creator, "err", aErr)
case validator != assigned:
log.Warn("[DIAG-1c] M2 MISMATCH (legacy→ErrFailedDoubleValidation)"
, "number", number, "creator", creator, "recovered", validator, "as
signedM2", assigned)
}
}
}
return nil
```

A header with number greater than the V2 switch does not use `verifySeal`. Mainnet switches at block 80,370,000 and Apothem at block 56,828,700. The creator still has to be in the signer set. When that holds, full verification imports the block with any `Validator` bytes.

Assets:

- XDPoS consensus [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

- In `verifySeal`, when `xdcFullVerify` is true and the number is past the first epoch, return `errNoValidatorSignature` for an empty `header.Validator`.
- Return `errFailedDoubleValidation` when `RecoverValidator` is not the M2 from `GetValidator`.
- Apply the same recovery check on `verifyHeaderLightV1` for mainnet, where the empty-field reject already runs.

F-2026-19621 - V1 Checkpoint Penalties Are Outside The Seal And Never Checked - Low

Description:

The V1 seal digest is `encodeSigHeader`. It covers the header through `Nonce`, and `BaseFee` when that field is set. It does not cover `header.Penalties`. `errInvalidCheckpointPenalties` is declared and never returned.

`verifyCascadingFields` asks the penalty hook for the addresses to subtract while it compares the checkpoint Extra list. It does not compare that hook result, or any previous checkpoint, with `header.Penalties`. The header RLP includes `Penalties`, so a different list is a different block hash. The Extra seal still recovers the same creator, and difficulty stays the same because difficulty is inside the digest.

Later epoch checkpoints call `removePenaltiesFromBlock` for each of the previous `LimitPenaltyEpoch` checkpoints. That constant is 4. The function loads the canonical header at that number and drops every address encoded in `header.Penalties` from the signer set used for the new checkpoint. Every node that has the same canonical header applies the same list.

The V2 proposer digest includes `Penalties`. This path is the V1 wrapper. A header with number greater than the V2 switch is verified on the V2 path. Mainnet switches at block 80,370,000 and Apothem at block 56,828,700. When a V1 checkpoint is the current head, `NewBlock` inserts a same-height sibling and `writeBlockAndSetHead` makes that sibling the head. `removePenaltiesFromBlock` then reads `header.Penalties` from the header stored at that number. `NewBlock` does not insert a checkpoint that is no longer the head.

The digest stops before `Penalties`:

```
enc := []interface{}{
header.ParentHash,
header.UncleHash,
header.Coinbase,
header.Root,
header.TxHash,
header.ReceiptHash,
header.Bloom,
header.Difficulty,
header.Number,
header.GasLimit,
header.GasUsed,
header.Time,
extraForHash,
header.MixDigest,
header.Nonce,
}
```

The next checkpoints read the stored field and drop those addresses:

```
penalties := extractAddressFromBytes(header.Penalties)
if len(penalties) == 0 {
return masternodes
}
return removeItemFromArray(masternodes, penalties)
```

The checkpoint still has to be the canonical header at that height, and the Extra seal still has to recover. When it does, the following four epoch checkpoints subtract whatever addresses are in that field.

Assets:

- XDPoS consensus [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 3/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

- Include `header.Penalties` in `encodeSigHeader`.
- In `verifyCascadingFields`, compute the expected penalty list from the penalty hook and return `errInvalidCheckpointPenalties` when `header.Penalties` differs from it.
- Keep `removePenaltiesFromBlock` reading that checked field.

F-2026-19640 - A Sealed Sign-Slot Block Submits A Second Signing Transaction - Low

Description:

After a node seals a block whose number is divisible by 15, `commitMinedBlock` inserts it and then calls `broadcastSigningTx`. That submits one zero-price `sign` transaction to `0x89` for that block hash. A V1 block with an empty `Validator` returns before the insert, so it does not reach this call.

`InsertChain` queues one canonical-head event when it returns. `chainHeadCh` is buffered, and the worker is still inside `commitMinedBlock`, so the event waits until that function returns. The head handler then runs `commitNewWork` and calls `maybeBroadcastImportedSignTx` for the same block.

That function is documented as the path for a head this node did not seal. It requires a coinbase, a block number divisible by 15, a V2 engine, a snapshot, and that coinbase in the snapshot candidate pool. It does not check that this worker just sealed the block. It calls `broadcastSigningTx` again.

The first submission is queued. `Add` is called with sync false, and `IsSigner` is never set, so the special-transaction fast path does not run. Promotion sets the pending nonce to the submitted nonce plus one after `Add` returns. The head handler runs `commitNewWork` before the import hook, so that promotion has finished. `signTxNonce` uses that pool nonce when it is at least the confirmed chain nonce. `CreateTransactionSign` therefore builds a second transaction at the next nonce for the same block hash. That nonce is different, so the pool does not reject it as a duplicate of the first.

`GetSigningTxCount` credits each masternode once for a referenced block hash. The extra transaction does not change the reward count or the penalty count. It spends one nonce and can be included in a later block. The import hook runs only while mining is on. A mining node that did not seal the head takes that hook once. A sealed block whose number is not divisible by 15 returns before either submission. A coinbase that is absent from the candidate pool still submits the first transaction and returns from the import hook.

The import hook calls the same broadcaster with no seal check:

```
// when a new canonical head arrives that we did NOT mint
func (w *XdcWorker) maybeBroadcastImportedSignTx(block *types.Block
) {
...
w.broadcastSigningTx(block)
}
```

The head event that reaches it is the one `InsertChain` queued for the block this worker just wrote.

The condition that must hold is a mining node, a sealed block that `InsertChain` accepts, a block number divisible by 15, a wired account manager and transaction pool, and a coinbase that is in the snapshot candidate pool. When it holds, the node submits two signing transactions for that block.

Assets:

- Mining / sync [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 1/5

Likelihood Rate: 3/5

Severity: Low

Recommendations

Remediation:

- In `maybeBroadcastImportedSignTx`, return when the block hash is the block `commitMinedBlock` just sealed.
- Keep the import hook for heads that arrived from a peer.

F-2026-19641 - NewBlock Imports A V1 Extension Without Recovering The Seal - Low

Description:

Every node starts `xdcSyncer`, and that start sets `XdcBulkSyncMode`. `xdcFullVerify` is false while that flag is set, and also while checkpoint sync has no state. `VerifyHeaders` then sends a V1 header through `verifyHeaderLightV1`. That function checks the parent link, a timestamp that is not in the future and is at least the parent time plus the period, the nonce, the vanity and seal lengths, an empty signer list on a non-checkpoint, checkpoint signer alignment, a zero mix digest, and an empty uncle hash. On mainnet it also rejects an empty `Validator` after the first epoch. Apothem accepts an empty `Validator`. The function does not recover the extra seal and does not call `verifySeal`.

`handleNewBlockXDC` forwards the packet to `NewBlock`. That handler inserts a block whose parent is the current head, and a same-height sibling whose parent is the head's parent. It does not read `xdcFullVerify`. `InsertChain` runs the light check, then checks the transaction root. When the node has state, the body must execute to the announced state root before the block becomes the canonical head. While checkpoint sync has no state, `Process` and state-root validation are skipped, and the block can still become the head. The extra seal bytes are not recovered, so no masternode key is required.

The light check returns before the seal:

```
if bulkSync {
err = c.verifyHeaderLightV1(chain, headers[i], headers[:i])
}
```

The gossip path that still inserts:

```
if block.ParentHash() != cur.Hash() {
// same-height sibling, or return
}
if _, err := h.chain.InsertChain(types.Blocks{block}); err != nil {
```

This applies while the local head is still a V1 block. Mainnet stays on that path until block 80,370,000 and Apothem until block 56,828,700. A block that does not extend the head and is not a same-height sibling is left for the syncer. A node for which `xdcFullVerify` is true uses `verifyHeader`, which ends in `verifySeal` and recovers the creator from the extra seal.

The condition that must hold is `xdcFullVerify` false, a V1 head, and a `NewBlock` whose parent is the current head or whose parent is the head's parent at the same height. The header must pass the light checks, and the body must execute to the announced state root. On mainnet, after the first

epoch, `Validator` must be non-empty. When those hold, that block becomes canonical without a seal the full check would reject.

Assets:

- XDPoS consensus [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

- In `verifyHeaderLightV1`, recover the V1 seal before accepting a block.
- While `XdcBulkSyncMode` is set, refuse `NewBlock` and let the syncer fetch the next header.

F-2026-19642 - Bulk Sync Accepts A V2 Block Dated In The Future - Low

Description:

`verifyHeader` rejects a V2 header whose timestamp is ahead of the local clock only when full verification is on. `xdcFullVerify` is false while `XdcBulkSyncMode` is set, and also while checkpoint sync has no state. Every node sets the bulk-sync flag when `xdcSyncer` starts. It stays set until the syncer records the node at the tip.

`InsertChain` verifies through that function. `NewBlock` inserts a block that extends the current head, or a same-height sibling, without reading `xdcFullVerify`. `fullSyncOnce` inserts the header batch it received on the same path. The future-time check is skipped. The parent spacing check still runs, except on the first block after the V2 switch: the header time must be at least the parent time plus the round's mine period. The header must carry a 65-byte proposer signature, a quorum certificate that verifies, a round greater than the certificate round, and a recovered signer that matches the coinbase. When the node has state, the body must execute to the announced state root before the block becomes the head. While checkpoint sync has no state, `Process` and state-root validation are skipped, and the block can still become the head.

A header that passes is stored under its hash. A later call for that hash returns before the future-time check, including after `xdcFullVerify` becomes true.

The next header is checked against the imported parent time. Spacing rejects it unless its time is at least the imported time plus the mine period. Once full verification is on, that next header is also rejected while its own time is still ahead of the local clock. A node for which `xdcFullVerify` is already true rejects the original header.

The check that full verification runs and bulk sync does not:

```
if fullVerify {
if header.Time > uint64(time.Now().Unix()) {
return consensus.ErrFutureBlock
}
}
```

The condition that must hold is `xdcFullVerify` false, a V2 header whose other `verifyHeader` checks pass, a timestamp ahead of the local clock, and a body that executes to the announced state root. When it holds, that block becomes the canonical head. A successor is not imported until its timestamp is at least one mine period later, and, once full verification is on, until the local clock reaches that later time.

Assets:

- XDPoS consensus [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

- Reject a V2 header whose `header.Time` is ahead of the local clock whether or not full verification is on.
- Do not cache a header that skipped that check.

F-2026-19644 - Apothem Protector Rewards Use Stake From The Payout Block - Low

Description:

Apothem sets `TIPUpgradeRewardBlock` to 83,600,000. Mainnet leaves that field unset, so `IsTIPUpgradeReward` is false there and this branch does not run. `HookReward` calls `GetSigningTxCountTiered` only when that gate is true. From round 27,360,000 the round config sets the protector cap to 3 and the observer cap to 2, and the configured amounts are 50.27 XDC and 25.13 XDC.

`GetSigningTxCountTiered` walks back two epoch switches. It takes the masternode list and the penalty list from that older checkpoint header. Protector and observer selection does not use that header's state. It loads candidates from `parentState`, which `Process` copies before the payout block's transactions, sorts them by stake, drops the zero address, the checkpoint penalties, and the checkpoint masternodes, and keeps the first three remaining addresses as protectors and the next two as observers. A stake change inside the payout block itself is not in that copy.

Signing credit is counted only on blocks divisible by 15, from the block after that checkpoint through the block before the epoch switch in between. An address in a tier that has at least one such credit is paid. `CalculateRewardForSignerFixed` gives each of those addresses the full configured tier amount. The sign count does not divide it. `GetRewardBalancesRateUpgrade` then assigns 90 percent to the candidate owner and 10 percent to the foundation. The voter share is zero. One address is not paid twice: masternodes are removed first, and protector versus observer is exclusive.

A stake change after that checkpoint and before the payout block's transactions can move an address across the cutoff. An address that was outside the standby set at the checkpoint, and that has a counted signature in the window above, is paid the protector or observer amount. The address that held the slot at the checkpoint is left out.

The ranking that uses the state at the start of the payout block:

```
nodesToKeep[MasterNodeBeneficiary] = c.GetMasternodesFromCheckpoint
Header(h, h.Number.Uint64(), chainConfig.XDPoS.Epoch)
candidates := state.GetCandidates(parentState)
```

The condition that must hold is an Apothem epoch-switch block at or past 83,600,000 whose round is at or past 27,360,000, and a stake change after the checkpoint header and before that block's transactions that changes who sits in the top three or the next two. The address that enters the tier must also have a counted signature in the window between the two

prior epoch switches. When it holds, the payout credits the configured protector or observer amount to the later ranking.

Assets:

- eth/hooks [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

- Rank protectors and observers from the candidate stake at the same checkpoint header that supplies the masternode list.
- Do not read `parentState` for that order.

F-2026-19724 - A Non-Mainnet Insert Error Clears Bulk Sync Away From The Tip - Low

Description:

`XdcBulkSyncMode` stays set while `xdcSyncer` is catching up. `xdcFullVerify` is false while that flag is set and while checkpoint sync has no state. The miner gate in `commitNewWork` also reads the flag. `StartMining` sets the separate synced gate, so a node that has started mining is held by the bulk-sync flag.

On a chain id other than 50, `fullSyncOnce` clears the flag when `InsertChain` returns an error other than `ErrUnknownAncestor` and `fromBlock` is at most the local head plus one. Mainnet, chain id 50, does not take this path. Apothem is chain id 51. Devnet is chain id 5551. Both do.

`fromBlock` is set to `localHead + 1` at the start of every round. After a failed insert that leaves the head at or above that starting number, `fromBlock <= head + 1` is true. The comment on the comparison describes a node that is already at the network tip, and a deep-history block that must not trip the clear. The value it reads is the start of the fetch, not the peer head. `ErrUnknownAncestor` is excluded. Any other insert error meets the test, including while the local head is still behind the network. `maybeRollbackOnUnknownAncestor` does not pause the miner for an error other than an unknown ancestor or a turn mismatch, and it runs after the clear.

Clearing the flag makes `xdcFullVerify` true when checkpoint sync still has state, so header verification becomes strict while the node is behind. It also opens the bulk-sync miner gate. A node that has started mining can then seal on the local head. A node that has not started mining does not seal. Nodes at the network tip reject a block that does not extend their head. A default node runs `fullSyncOnce` when the experimental snap-sync switch is unset.

The clear runs for every such error:

```
fromBlock := localHead + 1
```

```
if !errors.Is(err, consensus.ErrUnknownAncestor) &&
s.handler.chain.Config().ChainID != nil &&
s.handler.chain.Config().ChainID.Uint64() != 50 {
if cur := s.handler.chain.CurrentBlock(); cur != nil && fromBlock <
= cur.Number.Uint64()+1 {
if core.XdcBulkSyncMode.Load() {
core.XdcBulkSyncMode.Store(false)
}
}
}
```

The condition that must hold is a full-sync node whose chain id is not 50, and an `InsertChain` error other than an unknown ancestor, with the head

still at or above the number where the round started. When it holds, `XdcBulkSyncMode` is cleared while the local head can still be behind the network.

Assets:

- Mining / sync [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

- Clear `XdcBulkSyncMode` on this error only when the local head number is at the peer head the round just fetched.
- Keep `fromBlock <= head + 1` as the check that the failed batch started at the next local block. It does not show that the node is caught up.

F-2026-19774 - Voted Round, Mined Round And Lock Are Not Kept Across A Restart - Low

Description:

HotStuff safety depends on per-node state. `highestVotedRound` stops a masternode from voting twice in one round, `lockQuorumCert` stops it from voting for a block that conflicts with a certificate it already locked, and `highestSelfMinedRound` stops a leader from sealing twice in one round. The safety argument assumes a node still holds this state after a crash.

`sendVote` sets `highestVotedRound` in memory after the local vote handler accepts the vote, then broadcasts it. The value it stores is `currentRound`, read after the local handler returns, and that handler can complete a certificate through `processQC` and advance the round first (F-2026-19622). `processQC` raises `lockQuorumCert` in memory, and `Seal` sets `highestSelfMinedRound` in memory. None of the three values is written to the database. After a restart, `initial` reads the certificate in the head block and passes it to `processQC`, which rebuilds the lock from the certificate inside the certified block and sets `currentRound` to the certificate round plus one. `highestVotedRound` and `highestSelfMinedRound` start again at 0. `verifyVotingRule` passes any round above the voted round, and `yourturnAligned` allows any round above the mined round.

The voted round is set only in memory in `sendVote`:

```
x.highestVotedRound = x.currentRound

// Broadcast
x.broadcastToBftChannel(voteMsg)
```

The one-vote check in `verifyVotingRule` compares the current round with that value:

```
if x.currentRound <= x.highestVotedRound {
log.Info("[verifyVotingRule] Already voted this round",
"currentRound", x.currentRound,
"highestVotedRound", x.highestVotedRound)
return false, nil
}
```

A masternode stores block `B`, votes for it in round `r`, and restarts before round `r` ends. `B` is its head, and the certificate inside `B` is for the previous round, so `initial` returns the node to round `r` with `highestVotedRound` at 0. If the round-`r` leader also sent a conflicting block `B'`, the same-height sibling path in the `NewBlock` handler imports it and the node votes for `B'`. It has then signed two blocks in one round. The leader of round `r` is exposed the same way: after sealing `B` and restarting inside the round, it

returns to round `r` with `highestSelfMinedRound` at 0 and seals a second block for that round.

The double vote needs an equivocating leader and a restart inside the round; the double seal needs only the leader's own restart. Votes from honest nodes that restart are counted toward a certificate for the conflicting block, which lowers the number of faulty nodes needed to certify two blocks in one round. Separately, the rebuilt lock can be one round lower than the lock held before the restart when the node had processed a certificate for its head that no child block embeds yet. After such a restart, a proposal that extends the block below the certified head passes the lock rule in `verifyVotingRule`, where the lock held before the restart refused it. The voted-round latch alone stops the sibling vote above, so the lock regression is a separate weakening.

Assets:

- XDPoS consensus [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 3/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

- In `sendVote`, write the voted round to the database after `voteHandler` returns without error and before `broadcastToBftChannel`. A crash after the write and before the broadcast skips one vote; a crash after the broadcast and before the write allows a second vote.
- Store `blockInfo.Round`, not `currentRound`. `voteHandler` can run `processQC` and advance `currentRound` before the latch is assigned, and a persisted advanced value makes the node skip the next round after a restart.
- In `initial`, set `highestVotedRound` to the higher of the stored and in-memory values. `initial` also runs from `ReinitBFT` during sync, so overwriting the in-memory value can lower a live latch. The existing `currentRound <= highestVotedRound` check in `verifyVotingRule` then refuses that round.
- Persist `highestSelfMinedRound` the same way in `Seal`, before the sealed block is published, and in `initial` set it to the higher of the stored and in-memory values.

- As a separate fix, persist `lockQuorumCert` when `processQC` raises it, and in `initial` keep the higher of the stored and rebuilt certificates, since `processQC` installs the certificate from inside the head.

F-2026-19815 - Epoch-Switch Verification Accepts a Reordered Validator List - Low

Description:

An epoch-switch header carries the masternode list in `Validators`. Steady-state verification rebuilds that list with `calcMasternodes` and compares it with the header field through `CompareSignersLists`. The comparison copies both lists, sorts each by address bytes, and then checks equality. The same addresses in another order pass. That result does not enter the mismatch branch, so `fullVerify` applies no order check on this path.

The leader check on the epoch-switch block uses the `calcMasternodes` list in that list's order. While `fullVerify` is true, the only proposer `verifyHeader` accepts is the address at `round % Epoch % len` in that list. `Prepare` writes `Validators` by appending the same list. `Seal` signs `sigHash`, and `sigHash` includes `Validators`, so the signature covers the order in the header. The header RLP includes `Validators`, so that order is part of the block hash. A calculated leader who writes those addresses in another order still passes verification.

The accepted header is the list later blocks use. `GetMasternodesFromEpochSwitchHeader` copies `Validators` in byte order. `getEpochSwitchInfo` keeps that slice, and `GetMasternodes` returns it for a later header in the epoch. A non-epoch-switch block reads that stored list through `GetMasternodesWithParents`. Its leader is the address at `round % Epoch % len` in the stored list. For a round that is not an epoch switch, `yourturn` reads the same list. Every node that accepts the header reads the same bytes, so the network follows one rotation for the rest of the epoch: the order written in `Validators`.

The same addresses remain masternodes. That header order selects the leader of each later round.

`CompareSignersLists` sorts before it tests equality:

```
sort.Slice(l1, func(i, j int) bool {
return bytes.Compare(l1[i][:], l1[j][:]) == -1
})
sort.Slice(l2, func(i, j int) bool {
return bytes.Compare(l2[i][:], l2[j][:]) == -1
})
return reflect.DeepEqual(l1, l2)
```

The epoch record keeps the header order:

```
masternodes := make([]common.Address, len(header.Validators)/common
.AddressLength)
for i := 0; i < len(masternodes); i++ {
copy(masternodes[i][:], header.Validators[i*common.AddressLength:])
```

```
    }
    return masternodes
```

The condition is an epoch-switch header on the steady-state path, sealed by the leader `calcMasternodes` selects, whose `Validators` field is those same addresses in a different order, and whose `Penalties` field still matches under the same comparison. Verification accepts the header. Later blocks in that epoch take the leader from the header order. Nodes that share this code then select the same leader, because each of them reads `Validators` in byte order.

Assets:

- XDPoS consensus [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

- In `verifyHeader`, when `fullVerify` is true and `calcMasternodes` has returned a list, require `Validators` to equal that list in order. Return `ErrValidatorsNotLegit` when the order differs.
- Keep `CompareSignersLists` on the path where `fullVerify` is false. Later blocks already take the leader index from the stored header order, so the ordered check at the epoch switch is what keeps that rotation equal to `calcMasternodes`.

F-2026-19471 - FastSync Auto Pin Trusts Compiled Checkpoints Without Quorum Verification - Info

Description:

Public XDPoS chain configs embed `TrustedSyncCheckpoints`, a list of block number, hash, state root, and masternode addresses. Fast sync calls `AutoPinPivotFromCheckpoints` when the pivot number is still zero and auto-pin is enabled. The function copies the highest selected tuple's number, hash, and root into the pivot fields. It does not recover a quorum certificate, and it does not read an epoch-switch header.

The selector keeps a tuple only when the hash is non-zero, the root is non-zero, and the masternode list is non-empty. Those values come from the binary. The masternode addresses are a selection filter. They are not copied into the pivot.

Genesis setup writes the compiled list into an empty checkpoint list for chain IDs 50 and 51. Chain ID 5551 is in the same switch, and its compiled list is empty, so that assignment stays empty. Stored-config load and the genesis spec perform the same assignment.

`ValidateBody` skips `HasBlockAndState` when the parent number and hash match any compiled tuple. The comparison runs for every XDPoS body whose config carries the list. Header verification runs before this body check, and a header-verification error still rejects the block. The skipped body check is the one that rejects a missing parent and a parent block that has no local state.

While the pivot number is non-zero, state sync requests the compiled root. When state sync completes, a served pivot header whose hash differs from the compiled hash is rejected. That hash comparison does not require the compiled root to equal the root inside the served header.

Nodes that fast-sync with auto-pin enabled and the same compiled list pin the same number, hash, and root. A remote peer cannot replace the compiled hash. An operator can still supply an explicit pivot, or disable auto-pin. Later headers remain subject to engine header verification. The omitted check is quorum verification of the pivot before it is installed.

The selector drops a tuple that lacks a hash, a root, or a masternode list, then keeps the highest remaining number:

```
if cp.Hash == (common.Hash{}) || cp.Root == (common.Hash{}) {
continue
}
if len(cp.Masternodes) == 0 {
continue
}
if best == nil || cp.Number > best.Number {
best = cp
}
```

The pin writes three fields:

```
cfg.FastSyncPivotNumber = best.Number
cfg.FastSyncPivotHash = best.Hash
cfg.FastSyncPivotRoot = best.Root
```

Genesis setup fills an empty list from the compiled config:

```
if len(newCfg.TrustedSyncCheckpoints) == 0 && newCfg.ChainID != nil
{
switch newCfg.ChainID.Uint64() {
case 50:
newCfg.TrustedSyncCheckpoints = params.XDCMainnetChainConfig.Truste
dSyncCheckpoints
```

Body validation skips the on-disk ancestor check when the parent matches a compiled tuple:

```
if cp != nil && cp.Number == parentNum && cp.Hash == parentHash {
parentIsTrustedCheckpoint = true
break
}
if !parentIsTrustedCheckpoint && !v.bc.HasBlockAndState(parentHash,
parentNum) {
```

State sync uses the compiled root once a pivot number is set:

```
if d.operatorPivotNumber != 0 {
pivotRoot = d.operatorPivotRoot
}
sync := d.syncFastState(pivotRoot)
```

Completion rejects a different pivot hash:

```
if d.operatorPivotHash != (common.Hash{}) {
if P.Header.Hash() != d.operatorPivotHash {
return fmt.Errorf("fast sync: pivot hash mismatch — operator pinned
%s, peer served %s",
d.operatorPivotHash.Hex(), P.Header.Hash().Hex())
}
}
```

Assets:

- Core [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Severity: Info

Recommendations

Remediation:

- In `AutoPinPivotFromCheckpoints`, verify the pivot quorum certificate against the masternode set on the epoch-switch header before copying the compiled number, hash, and root into the pivot.
- Keep the compiled masternode list as a selection filter. Do not treat it as that quorum check.
- In `ValidateBody`, call `HasBlockAndState` for a parent that matches a `TrustedSyncCheckpoints` entry.
- When the served pivot header hash equals `operatorPivotHash`, pass that header's root to `syncFastState`. Do not replace it with `operatorPivotRoot`.
- In genesis setup, stored-config load, and the genesis spec, leave an empty `TrustedSyncCheckpoints` list empty unless the operator opts in.

F-2026-19615 - V1 Header Verification Does Not Bound The Gas Limit - Info

Description:

V2 header verification rejects a `GasLimit` that moves by more than `parent.GasLimit/1024 - 1`, a `GasLimit` below `params.MinGasLimit`, and a `GasUsed` above `GasLimit`. The V1 path does not. `verifyHeader` checks the timestamp, the checkpoint coinbase and nonce, vanity and seal length, the Extra signer alignment, a zero mix digest, and an empty uncle hash. `verifyCascadingFields` checks the parent link and the timestamp gap, then the checkpoint signer list and the seal. Neither function reads `GasLimit` or `GasUsed`.

`CalcGasLimit` is what the miner uses to pick the next limit. Import does not call it. Execution draws from a gas pool of `header.GasLimit`, and state validation compares `header.GasUsed` with the gas consumed. The body has to fit in the announced limit. The limit itself can change by more than the 1/1024 bound and can sit below `MinGasLimit`.

The `GasUsed > GasLimit` reject in the fallback branch of `verifyHeaderV2` is not reached. A wired engine returns from `verifyHeaderV2` before it, a missing engine returns the uninitialized-engine error before it, and a V1 block never enters `verifyHeaderV2`.

V2 enforces the bound in two checks, then rejects `GasUsed` above `GasLimit`:

```
diff := int64(parent.GasLimit) - int64(header.GasLimit)
if diff < 0 {
diff *= -1
}
limit := parent.GasLimit / params.GasLimitBoundDivisor // 1/1024 bo
und
if uint64(diff) >= limit {
return fmt.Errorf("invalid gas limit: have %d, want %d +/- %d", hea
der.GasLimit, parent.GasLimit, limit-1)
}
if header.GasLimit < params.MinGasLimit {
return fmt.Errorf("invalid gas limit below %d", params.MinGasLimit)
}
if header.GasUsed > header.GasLimit {
return fmt.Errorf("gas used exceeded gaslimit, gas used: %d, gas li
mit: %d", header.GasUsed, header.GasLimit)
}
```

A header with number greater than the V2 switch uses that V2 check. Mainnet switches at block 80,370,000 and Apothem at block 56,828,700. On the V1 path the in-turn seal still has to recover, and execution still has to fit the announced limit. The header is imported at whatever `GasLimit` it carries.

Assets:

- XDPoS consensus [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Severity: Info

Recommendations

Remediation:

- In `verifyCascadingFields`, reject a `GasLimit` outside `parent.GasLimit/1024 - 1` of the parent, a `GasLimit` below `params.MinGasLimit`, and a `GasUsed` above `GasLimit`.
- Use the same three checks the V2 header path already applies.

F-2026-19725 - Protector Ranking Uses A Stable Sort On Equal Stake - Info

Description:

Apothem sets `TIPUpgradeRewardBlock` to 83,600,000. From round 27,360,000 the round config sets the protector cap to 3 and the observer cap to 2. `HookReward` calls `GetSigningTxCountTiered` when that fork is on. The function loads candidates from `parentState`, orders them with `slices.SortStableFunc` by stake, drops the checkpoint penalties and the checkpoint masternodes, and keeps the first three remaining addresses as protectors and the next two as observers. An address in a tier with a counted signature is paid the full tier amount.

`UpdateMasternodesFromHeader` builds the masternode list with `xdc_sort.Slice` and the comparison `stake.Cmp >= 0`. That sort is the one the engine uses so that equal stakes keep the canonical order. The comment on that sort records that the standard library sort orders equal stakes differently, and that the difference changes who falls inside a top-N cutoff. `SortStableFunc` keeps the order returned by `GetCandidates` when two stakes compare equal. That order is not the `xdc_sort` order.

Mainnet leaves `TIPUpgradeRewardBlock` unset, so this branch does not run there. Seated masternodes are still taken from the checkpoint header. The tie changes only the protector and observer prefixes.

The reward order:

```
slices.SortStableFunc(ms, func(a, b utils.Masternode) int {
return b.Stake.Cmp(a.Stake)
})
```

The order used for the masternode cutoff:

```
xdc_sort.Slice(cs, func(i, j int) bool {
return cs[i].stake.Cmp(cs[j].stake) >= 0
})
```

The condition that must hold is an Apothem epoch-switch block at or after 83,600,000 whose round is at or after 27,360,000, and an equal stake among non-masternode, non-penalized candidates that crosses the protector cap or the observer cap. The address this sort places in the tier must also have a counted signature in the reward window. When it holds, the payout credits the tier amount to the address the stable sort kept, not to the address the canonical stake order would have kept.

Assets:

- eth/hooks [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Severity: Info

Recommendations

Remediation:

- In `GetSigningTxCountTiered`, order the candidates with the same `xdc_sort.Slice` comparison `UpdateMasternodesFromHeader` uses.

F-2026-19805 - Reward RPCs Do Not Return The Credited Epoch Reward - Info

Description:

`eth_getRewardByHash` and `xdc_getRewardByHash` look up a reward record for the block in `common.StoreRewardFolder`. `XDPoS_getRewardByAccount` reads `XDPoS.StoreRewardFolder`, a second variable. Those two names are package-level variables, and neither is ever set. `GetRewardByAccount` returns an error. `GetRewardByHash` returns an empty result when the record is missing.

On an epoch switch, finalization runs the reward hook. The hook credits the balances and returns the reward record. Finalization discards that record and stores the block. The only place a reward file is written is the older assembly path, which a current-tip block does not reach. An epoch switch therefore credits the balances and leaves nothing for any of the three methods to read.

A block that is not an epoch switch has no reward, so an empty result matches that block. An epoch-switch block whose hook credited balances is answered the same way. All three methods are registered on a default node. Import does not read the reward record.

The hash lookup returns an empty result when the record is missing:

```
return make(map[string]map[string]map[string]*big.Int)
```

Finalization discards the record the hook returned:

```
if x.HookReward != nil && isEpochSwitch {
log.Info("[Finalize] V2 HookReward firing", "block", header.Number.
Uint64(), "hash", header.Hash().Hex())
_, err := x.HookReward(chain, state, parentState, header)
```

The condition that must hold is a call for an epoch-switch block on which the reward hook credited balances. When it holds, the response is empty or an error, and those balances are already in state.

Assets:

- RPC [https://github.com/XDCIndia/xdc-geth-audit]

Status: Pending Fix

Classification

Severity: Info

Recommendations

Remediation:

- Keep the map `HookReward` returns and serve `GetRewardByHash` and `GetRewardByAccount` from that record.
- Read one shared store from both methods. Do not require a folder variable that nothing assigns.

Observation Details

Disclaimers

Hacken Disclaimer

The blockchain protocol given for audit has been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in the protocol source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other protocol statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of the blockchain protocol.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.
The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at https://hacken.io/audits/.

Technical Disclaimer

Blockchain protocols are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the protocol can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited blockchain protocol.

Appendix 1. Severity Definitions

Severity	Description
Critical	Vulnerabilities that can lead to a complete breakdown of the blockchain network's security, privacy, integrity, or availability fall under this category. They can disrupt the consensus mechanism, enabling a malicious entity to take control of the majority of nodes or facilitate 51% attacks. In addition, issues that could lead to widespread crashing of nodes, leading to a complete breakdown or significant halt of the network, are also considered critical along with issues that can lead to a massive theft of assets. Immediate attention and mitigation are required.
High	High severity vulnerabilities are those that do not immediately risk the complete security or integrity of the network but can cause substantial harm. These are issues that could cause the crashing of several nodes, leading to temporary disruption of the network, or could manipulate the consensus mechanism to a certain extent, but not enough to execute a 51% attack. Partial breaches of privacy, unauthorized but limited access to sensitive information, and affecting the reliable execution of smart contracts also fall under this category.
Medium	Medium severity vulnerabilities could negatively affect the blockchain protocol but are usually not capable of causing catastrophic damage. These could include vulnerabilities that allow minor breaches of user privacy, can slow down transaction processing, or can lead to relatively small financial losses. It may be possible to exploit these vulnerabilities under specific circumstances, or they may require a high level of access to exploit effectively.
Low	Low severity vulnerabilities are minor flaws in the blockchain protocol that might not have a direct impact on security but could cause minor inefficiencies in transaction processing or slight delays in block propagation. They might include vulnerabilities that allow attackers to cause nuisance-level disruptions or are only exploitable under extremely rare and specific conditions. These vulnerabilities should be corrected but do not represent an immediate threat to the system.

Appendix 2. Scope

The scope of the project includes the following components from the provided repository:

Scope Details	
Repository	https://github.com/XDCIndia/xdc-geth-audit
Commit	45e2b6ee2b21bb2c8aa05ef5927695c513008cc3

Components in Scope

Consensus

- consensus/XDPoS/xdpos.go
- consensus/XDPoS/engines/engine_v2/engine.go
- consensus/XDPoS/engines/engine_v2/verifyHeader.go
- consensus/XDPoS/engines/engine_v2/vote.go
- consensus/XDPoS/engines/engine_v2/timeout.go
- consensus/XDPoS/engines/engine_v2/utils.go
- consensus/XDPoS/engines/engine_v2/epochSwitch.go
- consensus/XDPoS/engines/engine_v2/snapshot.go
- consensus/XDPoS/engines/engine_v2/mining.go
- consensus/XDPoS/engines/engine_v2/forensics.go
- core/types/consensus_v2.go
- consensus/XDPoS/network_constants.go
- params/config.go
- consensus/XDPoS/penalty.go
- consensus/XDPoS/reward.go
- eth/hooks/engine_v2_hooks.go
- eth/util/util.go
- core/state/statedb_utils.go
- contracts/utils.go
- eth/bft/handler.go
- miner/xdpos_worker.go

Sync paths

- eth/sync_xdc.go
- eth/handler.go
- eth/handler_eth.go
- eth/handler_xdc.go
- eth/downloader/downloader.go
- eth/downloader/faststatesync.go
- eth/downloader/gap_pivot_xdc.go
- eth/downloader/pivot_freeze_xdc.go
- eth/downloader/snap_terminate_floor_xdc.go
- eth/downloader/skeleton.go
- eth/downloader/beaconsync.go

- eth/downloader/peer.go
- eth/downloader/queue.go
- eth/downloader/syncmode.go
- eth/ethconfig/auto_pivot_xdc.go
- eth/ethconfig/config.go
- consensus/XDPoS/engines/engine_v2/snap_pivot.go
- eth/protocols/snap/xdc_snap_verify.go
- eth/protocols/eth/protocol.go
- eth/protocols/eth/handshake.go
- eth/protocols/eth/handlers.go
- eth/protocols/eth/peer.go
- p2p/discover/v4wire/v4wire.go
- params/bootnodes.go
- core/xdc_sync_opts.go
- core/blockchain_heal_status_xdc.go

Core / blockchain (transactions, blocks, state)

- core/types/block.go
- core/types/transaction.go
- core/types/state_account.go
- core/blockchain.go
- core/blockchain_reader.go
- core/blockchain_xdc.go
- core/block_validator.go
- core/state_processor.go
- core/state_transition.go
- core/txpool/legacypool/legacypool.go
- core/txpool/validation.go
- core/state/statedb.go
- core/state/state_object.go
- core/genesis.go
- core/genesis_xdc.go
- core/rawdb/chain_freezer.go
- core/rawdb/accessors_sync.go
- core/rawdb/schema.go
- core/txindexer.go

Appendix 3. Additional Valuables

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment, and the Penetration Testing Execution Standard (PTES), These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.